

Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Kacper Jednaszewski

29th April 2025

Implementation of a Hardware Accelerator for
Cryptographic Applications.

Project supervisor: Basel Halak
Second examiner: Sarah Morris

A project report submitted for the award of
Meng Aerospace Electronic Engineering

Abstract

Quantum Computers pose a serious threat to modern day cryptography with the creation of quantum algorithms like Shor's algorithm [6][7] that can potentially solve the mathematical problems, modern cryptosystems receive their security from, in a fraction of the time it takes classical computer [7]. This could lead to vital infrastructure being compromised and as a result NIST (National Institute of Standards and Technology) has tasked experts to help develop and evaluate a set of new standards that are resilient to attacks by quantum computers. In 2024, NIST announced three new standards [9] that should be implemented from this point on FIPS 203[10], 204 and 205, this project will focus on implementing a SHA-3 core utilized in FIPS 203 with the goal to accelerate it. The project developed 2 main hardware implementations seeking to minimize clock cycles. As the project resulted in an implementation with a significant clock cycle reduction in addition to providing a more approachable introduction to cryptosystems this project can be deemed as a moderate success.

Statement of Originality

Statement of Originality

- I have read and understood the [ECS Academic Integrity](#) information and the University's [Academic Integrity Guidance for Students](#).
- I am aware that failure to act in accordance with the [Regulations Governing Academic Integrity](#) may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have not used any resources produced by anyone else.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before reusing any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Acknowledgements

I would like to thank the university staff and especially Dr Basal Halak, who supervised this project and was helpful with his regular guidance.

Contents

Abstract.....	ii
Statement of Originality.....	iii
Acknowledgements.....	iv
Contents	v
1 Introduction.....	1
1.1 Goals and Objectives	1
1.2 Report Structure	1
2 Literature Review	2
2.1 Modern Cryptography.....	2
2.2 Quantum Computing Impact on Cryptography	2
2.3 NIST's New Standard.....	2
2.4 FIPS 203	3
2.5 Hashing Algorithms.....	4
2.6 SHA-3.....	5
2.7 SHA-3 Core	5
2.7.1 Delta.....	6
2.7.2 Rho.....	7
2.7.3 Pi	7
2.7.4 Chi.....	7
2.7.5 Lota	8
2.8 Previous Work	9
2.8.1 Keccak Team	9
2.8.2 Hardware acceleration design of the SHA-3 for high throughput and low area on FPGA.....	9
2.8.3 Pipelining SHA-3.....	10
3 Design Strategies	11
3.1 Development Summary	11
3.2 Hashing in c	11
3.3 Approach.....	11
3.4 Top Level File.....	13
3.5 Delta.....	13
3.5.1 Implementation 1	13
3.5.2 Implementation 2	14
3.6 Rho.....	14
3.6.1 Implementation 1	14
3.6.2 Implementation 2	14
3.7 Pi.....	14
3.7.1 Implementation 1	14
3.7.2 Implementation 2	14
3.8 Chi.....	15
3.8.1 Implementation 1	15
3.8.2 Implementation 2	15
3.9 Lota	15
4 Testing and Evaluation	17
4.1 Testing Functionality	17
4.1.1 Test Bench Construction.....	17

4.2	Evaluation of Clock Cycles.....	18
4.3	Evaluation of Approach	18
4.4	Waveform Screenshots.....	19
4.4.1	Delta	19
4.4.2	Rho	20
4.4.3	Pi.....	21
4.4.4	Chi.....	22
4.4.5	Lota.....	22
5	Conclusions	23
5.1	Project Management.....	23
5.2	Future work	23
5.2.1	Original Gantt Chart.....	25
5.2.2	Final Gantt Chart.....	26
	References	28
Appendix A.	C code.....	31
Appendix B.	Code for implementation 1.....	35
Appendix C.	Code for Implementation 2	48
Appendix D.	Header Files.....	59

1 Introduction

The rise of Quantum computing research and the threat they pose to existing cryptographic standards has caused either total vulnerability or required an accommodation in key size. In response NIST (National Institute of Standards and Technology) has worked with experts to develop new standards that are resistant to both quantum and classical computers. This process has resulted in 3 new standards, FIPS (Federal Information Processing Standards) 203, FIPS 204 and FIPS 205. The focus of this project will be on the family of cryptographic hashing algorithm found in FIPS 203, SHA-3 as discussed in [17], hashing algorithm are vital for the operation and security of the whole cryptosystem due to their wide range of uses including but not limited to the truncation of variables concatenated together for example keys. They also find potential usage in dispersing entropy inside a random number generator. This along with the relative simplicity of the algorithm made it a good candidate for the focus of this project and acceleration.

1.1 Goals and Objectives

The main objective of this project will be the implementation of the SHA-3 core in system Verilog; the design and general outline of this implementation will follow the guidance of [17] which details the general structure of the algorithm as well as providing useful illustration of effects its sub algorithms have on the bit string. Once a rudimentary design has been implemented, the goal will be to create further implementations that will seek to improve upon and accelerate the previous implementation with the ultimate goal of decreasing the total clock cycles.

A secondary goal of the project will be to create a guide for those seeking to create/develop their own implementations of SHA-3 as well as acting as a framework for accelerating the algorithm by providing an in-depth description of the steps that were taken when working on this project.

Finally, this project will document a series of test that have been conducted on said implementations including a functionality test against a software implementation that will be used as a source for the test vectors. Additionally, an evaluation of the implementations with an analysis of the pros and cons of each implementation will be conducted.

1.2 Report Structure

The report features a literature review in section 2, this provides all the relevant information needed to understand the design as well as providing essential context about the importance of cryptographic hashing algorithms in cryptosystems. In addition, it covers previous work conducted involving the SHA-3 family of algorithms. Section 3 details the design of the algorithms describing the differences between the implementations as well as justifying their existence. Section 4 includes the functionality tests that were conducted as well as evaluation of the two systems. Lastly, section 5 includes a summary of the project, a detailing of future work that can be done with the design that I have provided as well as a description of the project management.

2 Literature Review

2.1 Modern Cryptography

Communication over a public channel occurs constantly throughout modern society in every industry due to how prolific the internet has become with 59% surveyed users saying they could not imagine their lives without the internet and 20% saying they don't feel secure saving their data online [2]. Additionally various data breaches have costed the UK millions a year [3] and therefore there is a vested interest and need to protect the data being send over these public channels to ensure that confidential data remains confidential. Cryptography acts as a tool to achieve this confidentiality by suppling two parties with a key that allows data to be "locked" and "unlocked" masking the actual information behind a seemingly incoherent message.

Cryptographic algorithms come in two forms, symmetric and asymmetric, however the focus will be on symmetric algorithms as that is what is being impacted in this project. Symmetric algorithms work off the basis of two parties sharing a secret key that is unique to the pair that allows them to decrypt and encrypt messages that have been sent between the pair this key will be identical for both parties and there will need to be some way to share said key securely this is the crux of the symmetric cryptosystem [1].

2.2 Quantum Computing Impact on Cryptography

Quantum computers exist as a mixture of quantum mechanics and classical computers, like classical computers they have registers and gates however the main distinction is that instead of bits they use qubits which exhibit quantum behaviours. This quantum behaviour allows a qubit to exist as both a one and zero simultaneously as well as every value in between this allows a single computation using qubits to explore multiple inputted values [4]. However, there are certain issues due to this quantum behaviour namely a lack of reliability, this quantum state can be affected by environmental factors resulting in the state of the qubit being inconsistent and as such error correction is needed which can in turn be unreliable [5]. Regardless of this unreliability, the assumption is that quantum computers will be usable some time in the future and as such their effect on current standards need to be considered.

Cryptosystems rely on time consuming mathematical problems like RSA [1], AES and elliptic curve [6] however it can be shown using Shor's quantum algorithm [6] that cryptosystems like RSA that rely on prime factorisation [7], can be broken in a fraction of the time it takes a classical computer to do the same, due to a deficiency in efficient classical algorithm for factorising primes [7]. As a result, quantum computers pose a serious threat to established cryptosystems and as such new cryptographic algorithms need to be designed to accommodate this threat.

2.3 NIST's New Standard

In response to the future threat that quantum computers posed on cryptosystems NIST (the National Institute of Standards and Technology) asked experts to help develop and evaluate new post quantum cryptographic algorithms with the aim to standardise a set of

algorithms that are resilient against attacks using quantum computers [8]. This process resulted in 3 new standardized algorithms FIPS 203, 204 and 205. FIPS 203 is a KEM scheme that distributes a shared secret key and the other two are digital signature schemes. [9] With the standardisation of these new algorithms NIST also released three documents detailing the three standards as well as providing recommendations when it come to the design of the system to ensure a certain level of security.

2.4 FIPS 203

FIPS 203 is one of the new standards that NIST announced in 2024, and it is a KEM (Key Exchange Mechanism) the standard describes a set of algorithms that aim to establish a shared key between two parties [10]. In general, KEM is a set of three core algorithms that create a decapsulation and encapsulation key and then use said keys in two separate algorithms to distribute a shared secret key as well as verify that both parties have the same secret key as can be seen in figure 1.

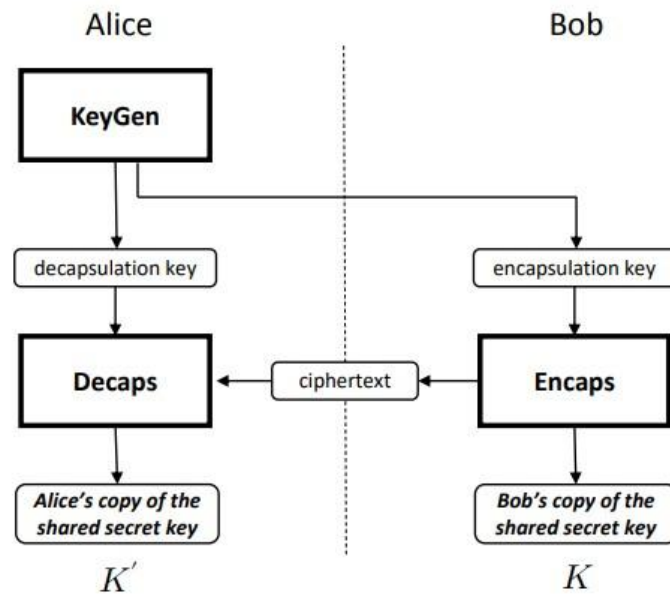


Figure 1 as seen in [10] shows the basic structure of KEM

During the process algorithm KeyGen is run which generates a private decapsulation key as well as a public encapsulation key, Alice sends this encapsulation key to Bob who uses said key to generate a shared secret key as well as the ciphertext which contains the secret key. Bob send this ciphertext text to Alice who uses her private decapsulation key to retrieve her copy of the secret key she can use a local copy of her decapsulation key to repeat the process and compare the output shared key she retrieved local to the one she retrieved from Bob to ensure that both parties have the same key.

The strength of this algorithm is based on the assumed strength of the Module Learning with Error problem which is a derived from the Learning with Error problem [11]. This problem is also extremely similar to the learning parity with noise problem, they all rely on the idea of solving for a hidden vector when given a set of noisy linear equations and more importantly they don't succumb to quantum algorithms and as such are appropriate to use as a basis for a PQC algorithm [12]. The problem states that given a

matrix A it is difficult to find a secret vector s that approximates a vector u given that there is some random error introduced to s , or in the form of a equation (1) [13].

$$(A \cdot s + e) \bmod p = u \quad (1)$$

This problem forms the basis for the public key encryption scheme used in FIPS 203 which generates the encapsulation key, this is the same as the encryption key, as well as the decapsulation key, which is a concatenation of the decryption key, a hashed encryption key, the encryption key and some randomness Z . In a sense A and u forms the encryption key and s represents the decryption key.

2.5 Hashing Algorithms

Hashing algorithms are almost ubiquitous in a variety of computing applications due to their unique set of properties that make them appealing for various applications. A hashing function seeks to map pieces of data into a hash table as efficiently as possible, this is characterised by the creation of a hashing algorithm that takes an arbitrary piece of data and assigns it a unique position or bucket inside a hash map. This unique bucket will only be accessible if the piece of data that was used to assign the bucket is known, think of a bank with a series of identical vault doors the piece of data used to assign the bucket will be the only way to access the bucket just like a key to a specific vault. To achieve this a hashing function must ensure a certain set of rules which include collision resistance. The idea that if the same data is used twice, it will always return the same bucket and alternatively two unique data values will always return different buckets. However, this isn't feasible as it would require an infinitely large hash map and as such functionality is added such that when collisions happen, which will become more common the larger the set of inputs, there is a remedy, this can include using linear probing or separate chaining [21]. [22]

These properties make a hashing algorithm particularly useful in cryptography as you can obscure sensitive data such that it still holds its integrity (the hashed data will be unique) leading to the development of cryptographic hashing algorithms. Hashing algorithms take an unknown sized input and produce fixed length outputs, a bucket address. Imagine a file that has a significant size, by passing it through a cryptographic hashing function we can reduce its size to a fixed length, while simultaneously the bucket address which can only be obtained via the knowledge of the input, maintains its importance. This property essentially acts as a form of compression, other properties of cryptographic hashing algorithms include irreversibility, stating that given the output of the algorithm you wouldn't be able to find the input. Bit sensitivity, which states that similar inputs should have significantly different output and as mentioned above the reproducibility of a hash output given the same input [22].

Cryptographic hashing algorithms take a variable length input and return a fixed length output the majority of these algorithms achieve this by compressing the variable length input into a fixed length before it enters the cryptographic hashing algorithm this can be achieved by using a faster hashing algorithm [23]. The hashing process is then iterated multiple times which combined with the property of small change in input leading to large change in output causes the avalanche effect. The avalanche effect describes the phenomenon of repeated passes through a hashing algorithm completely changing the output increasing the hashes security. This idea is present in SHA-3 [17] during the *lota* step alters the matrix based on the iteration round.

2.6 SHA-3

Although other families of cryptographic hashing algorithms exist, for example MD5 [25], this project will focus on the SHA family of algorithms.

SHA, released by NIST, originated in 1995 with the introduction of SHA-1 [26] as stated before it was based around an iterative model similar to MD5 known as Merkle Damgard's structure [24]. The scheme takes in variable length data as an input and output's a compressed fixed length of 160 bits, the input is subjected to a process of padding where the input is scaled to be 64 bits long, after which it is appended by the original input length being described by an additional 64 bits. The original input is then split up into 512-bit blocks. Next chaining variables are initialized, and the 512-bit blocks are processed [26], although SHA-1 has seen a wide range of use due to emerging security concerns NIST has started phasing out the algorithm with usage fully ending in 2030 [27].

SHA-2 released in 2002 [28] and is very similar to SHA-1 with the key distinction that SHA-2 incorporates non-linear algorithms when compressing the input which provides a higher level of robustness when compared to SHA-1. Additionally the standard included functionality to allow for outputs of size 224-, 256-, 384- and 512-bit lengths [26].

After SHA-1 was discovered to be vulnerable, a competition begun in order to create a new standard SHA-3 this resulted in Keccak being developed and announced as the winner of competition in 2012 [17][20]. Due to SHA-2's prevalence in cryptosystems and its status as a hashing algorithm that is still secure, as opposed to SHA-1 there is a sense of inertia when it comes to transitioning to SHA-3 especially due to its slow performance in software and similar level of security even if SHA-3 is generally more secure due to its sponge architecture [29]. However, where SHA-3 really shines is in hardware and as such as more work is done implementing and improving SHA-3's hardware performance its use will increase.

2.7 SHA-3 Core

Although SHA-3 as documented in [17] features a sponge structure, for the relevance of this project this shall be ignored, and the core shall now be described.

The SHA-3 core is made of 6 algorithms that hash a 1600-bit fixed length bit string, the bit string is first processed into a 3d state matrix with dimensions of $5 \times 5 \times 64$ as seen in figure 2, where each element of the state matrix is described by the expression $5(x+5y)+z$ [17]

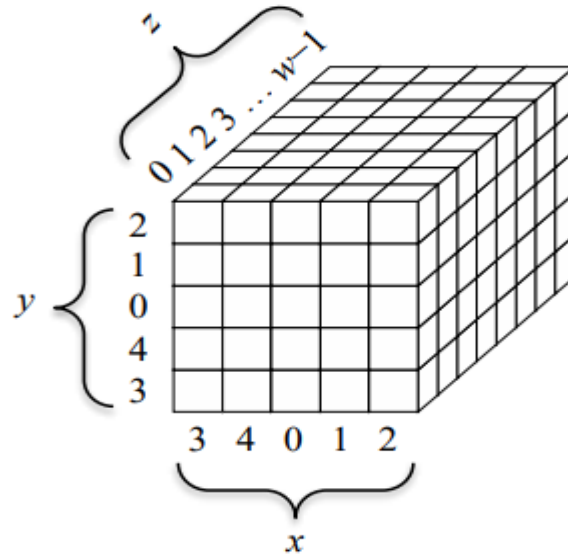


Figure 2: showcasing the state matrix and how it is indexed as seen in [17].

Once the bitstring has been processed, the algorithm then runs the matrix through a series of algorithms (theta, rho, pi, chi and lota) for 24 iterations assuming the width is 64 bits as specified in [17].

2.7.1 Delta

The first algorithm is always the delta function, it sum all the bits in all the column's using a XOR, essentially forming a plane from the state matrix, each bit is then subjected to a series of XOR operations by shifted bit's in the state matrix as seen in figure 3, the effect of which is to distribute the bits throughout the matrix this is a means to increase bit sensitivity [17]. The figure shows which columns relatively affect each other and the process is circular so elements with x coordinate 4 are affected by those in column 0.

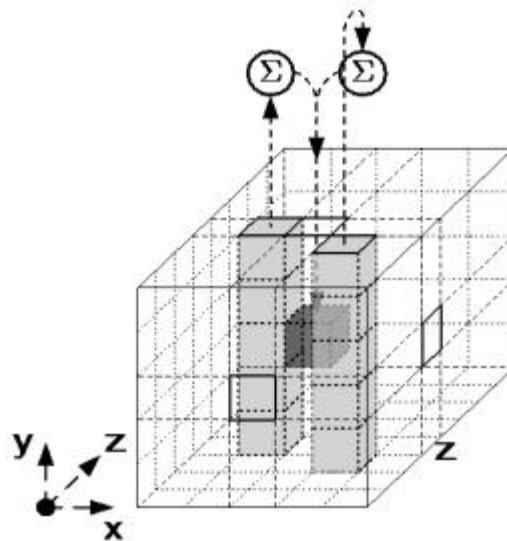


Figure 3, Showcasing the effect delta has on 1 bit as seen in [17].

The algorithm can be described using the following pseudocode in figure 4.

```

C[x] = A[x,0] xor A[x,1] xor A[x,2] xor A[x,3] xor A[x,4],    for x in 0...4
D[x] = C[x-1] xor rot(C[x+1],1),                             for x in 0...4
A[x,y] = A[x,y] xor D[x],                                     for (x,y) in (0...4,0...4)

```

Figure 4, showcasing the pseudocode describing delta as seen in [30]

2.7.2 Rho

Rho takes all the bits in a lane (think of the z axis of the state matrix) and offsets them by a certain value the offsets of each value can be calculated in real time or before run time and are simply adjusted using a modulo operation to fit the width of the lane. This is described in [17] as a rotation it is showcased in figure 5, which demonstrates the effect of the offset with the grey squares representing where the bit at $z = 0$ ends up.

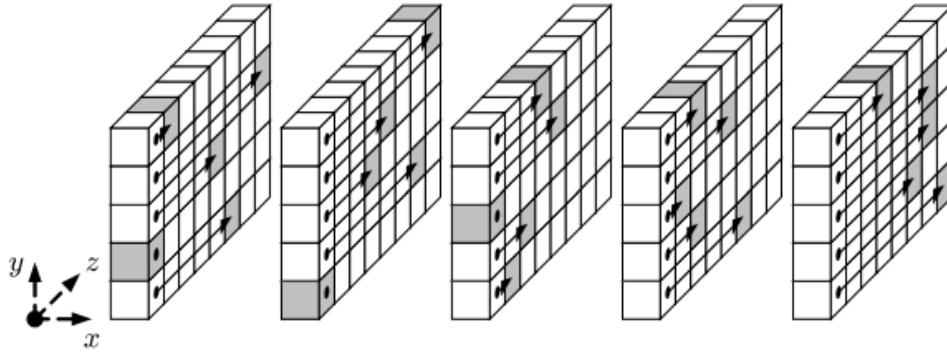


Figure 5, showcases the effect of the offset on all the lanes, as seen in [17].

2.7.3 Pi

Pi then rotates the lanes that have just been offset by Rho, as described in Figure 6, the effect of this algorithm is very similar to Rho, both rearrange the relative positions of the bits and as such similar to Delta both increase the dispersion of the bits increasing bit sensitivity over many iterations.

For all triples (x, y, z) such that $0 \leq x < 5$, $0 \leq y < 5$, and $0 \leq z < w$, let
 $A'[x, y, z] = A[(x + 3y) \bmod 5, x, z]$.
Return A' .

Figure 6, Showcases the algorithm used to rearrange the lanes, as seen in [17].

2.7.4 Chi

Chi introduces a non-linear element to the hashing algorithm by essentially multiplying two bits in a row and then applying an XOR to a bit in the same row, the algorithm is independent of the z coordinate. The algorithm is illustrated as a single operational row in figure 7.

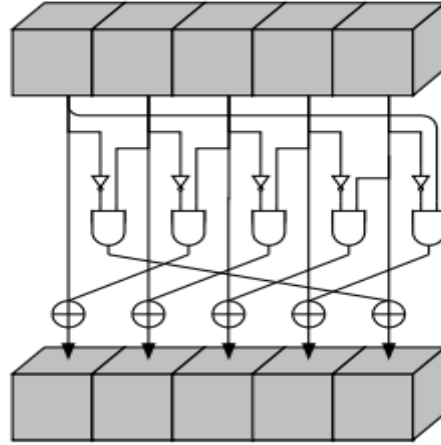


Figure 7, showcases the Chi algorithm applied to a singular row, as seen in [17]

2.7.5 Lota

Lota is split into two main sections a rc generator and the algorithm itself which applies a XOR to specific bits in the ($x=0$, $y=0$) lane. The rc generator is a pseudorandom linear shift register with a set seed of 00000001, the seed is then shifted based on the current iteration as well as the width of the state matrix. the most significant bit is then returned by the register. The bit is stored in a separate array which is the length of a lane and the only bits in said array that can be affected by the rc generator are the elements 0,1,3,7,15,31,63 this forms the basis for a simplification that is made when implementing the lota function. The rc generator is described in figure 8, while its effects on the state matrix are described in figure 9, this algorithm makes the state matrix directly dependant on the iteration number. This element of randomness while deterministic essentially provides an element of non-reversibility which is dependant on the width of the state matrix.

```

If  $t \bmod 255 = 0$ , return 1.
Let  $R = 10000000$ .
For  $i$  from 1 to  $t \bmod 255$ , let:
  a.  $R = 0 \parallel R$ ;
  b.  $R[0] = R[0] \oplus R[8]$ ;
  c.  $R[4] = R[4] \oplus R[8]$ ;
  d.  $R[5] = R[5] \oplus R[8]$ ;
  e.  $R[6] = R[6] \oplus R[8]$ ;
  f.  $R = \text{Trunc}_8[R]$ .
Return  $R[0]$ .

```

Figure 8, describes the rc generator in pseudocode, as seen in [17].

```

For all triples  $(x, y, z)$  such that  $0 \leq x < 5$ ,  $0 \leq y < 5$ , and  $0 \leq z < w$ , let  $A'[x, y, z] = A[x, y, z]$ .
Let  $RC = 0^w$ .
For  $j$  from 0 to  $\ell$ , let  $RC[2^j - 1] = rc(j + 7i_r)$ .
For all  $z$  such that  $0 \leq z < w$ , let  $A'[0, 0, z] = A'[0, 0, z] \oplus RC[z]$ .
Return  $A'$ .

```


Figure 9, describes the effect the rc generator values have on the state matrix, as seen in [17].

2.8 Previous Work

2.8.1 Keccak Team

As mentioned above, the creators of Keccak developed multiple implementations in VHDL, which can be found in [20], this included a fast but large implementation, slow but small implementation and lastly an implementation somewhere in between. These formed the basis for others works when developing a hardware implementation and as the code has been made open source it allows for modification and improvements to be made more easily.

Additionally, this framework of developing a large but quick and a small but slow implementation has been attempted to be adopted in this project as will be discussed in section 2.

2.8.2 Hardware acceleration design of the SHA-3 for high throughput and low area on FPGA

The usual architecture of a full Keccak system involves 1 full round of permutations in between the sponge functions however [31] proposes an unrolled architecture where 2 sets of permutations are conducted in between the sponge functions as seen in figure 10. This essentially halves the number of clock cycles greatly decreasing the latency of the system.

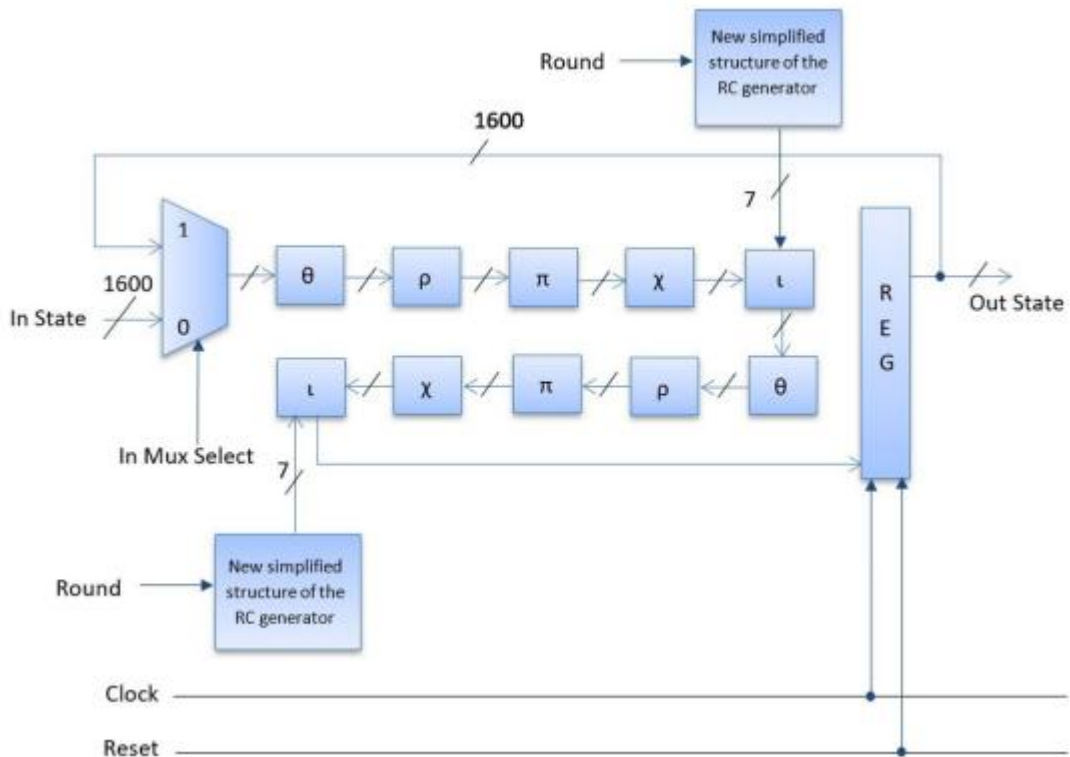


Figure 10, showcasing the effect of unrolling the algorithm, as seen in [31].

Additionally, the paper proposes a simplified rc generator, as mentioned above the generated values can only effect 7 bits in the lane that ι operates on. As such rather than applying a XOR on the whole lane computing it on only the bits that can be

affected greatly reduces the number of XOR operations that are conducted whenever *lota* is called.

2.8.3 Pipelining SHA-3

One method of pipelining SHA-3 is through the addition of multiple cores with multiplexers as discussed in [32], 2:1 multiplexer's feed into each core where one line acts as input and the other as a feedback. This allows the system to run multiple hashes simultaneously as each core handles a new permutation and once the permutation is finished it moves on to the next core, this is showcased by figure 11.

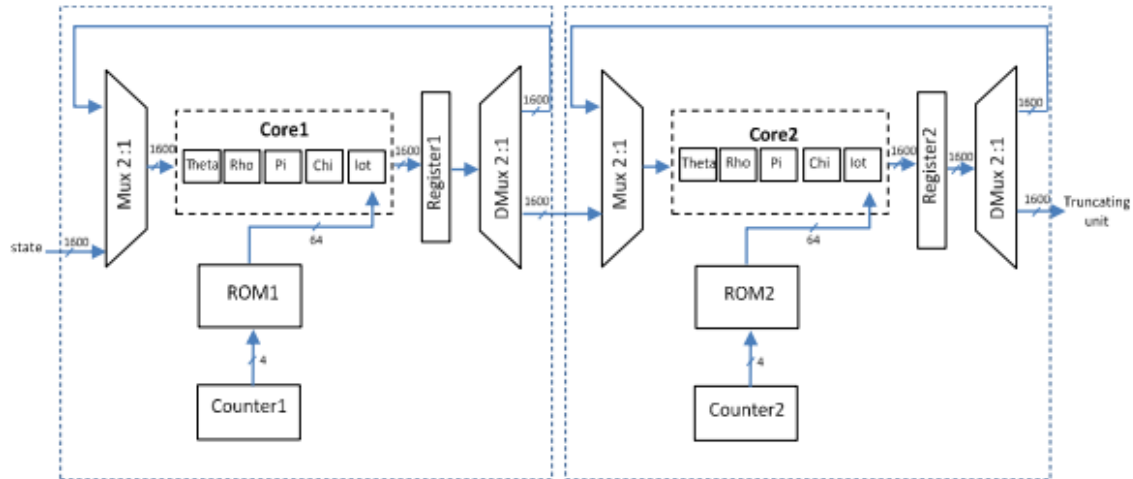


Figure 11, showcasing a pipelined SHA-3 core with 2 cores, as seen in [32].

3 Design Strategies

3.1 Development Summary

Three main implementations were worked on, one software implementation followed by two hardware implementations, the software implementation won't be analysed in this report and its main purpose was for testing purposes. The two hardware implementations follow two different philosophies the first implementation acts almost like a control, whereas the second implementation is my attempt at an accelerated algorithm. The details of which shall be discussed below.

3.2 Hashing in c

For testing purposes, as well to increase my understanding of the system initially, I decided to create a software implementation of the algorithm. I chose to develop the program in c first for two reasons c code in general is more readable than System Verilog and two I can use the program to generate test vectors speeding up the process of verifying the functionality of the System Verilog code.

To aid in its use as a tool for testing the hardware implementations functionality I modularised the program, this meant that every algorithm that made up the core had multiple breakpoints in a sense, so that I knew the expected state of the matrix after every alteration. Once I had finished each algorithm, I manually checked the output against the standards design and verified its functionality, once the c program had been completed, I started work on the System Verilog. The c code developed can be seen in Appendix A.

3.3 Approach

As seen in other implementations the system is usually split apart into its separate algorithms, I chose to follow a similar approach as it not only made the problem more streamlined by decomposing the problem, but it also made testing easier as errors would be isolated to the file they affected.

I chose to first develop a more literal approach to the algorithms and how they were described in [17], the approach saw the use of counters that would keep track of the algorithms position in the process of each alteration, rather than completing a full function in a clock cycle each individual bit alteration was given a separate cycle. This would mean that each individual clock cycle itself would be quick by there would be close to a hundred per function additionally as only individual shifts were occurring the size of the registers performing the shifts would be small. An illustration of the design approach can be seen in figure 12

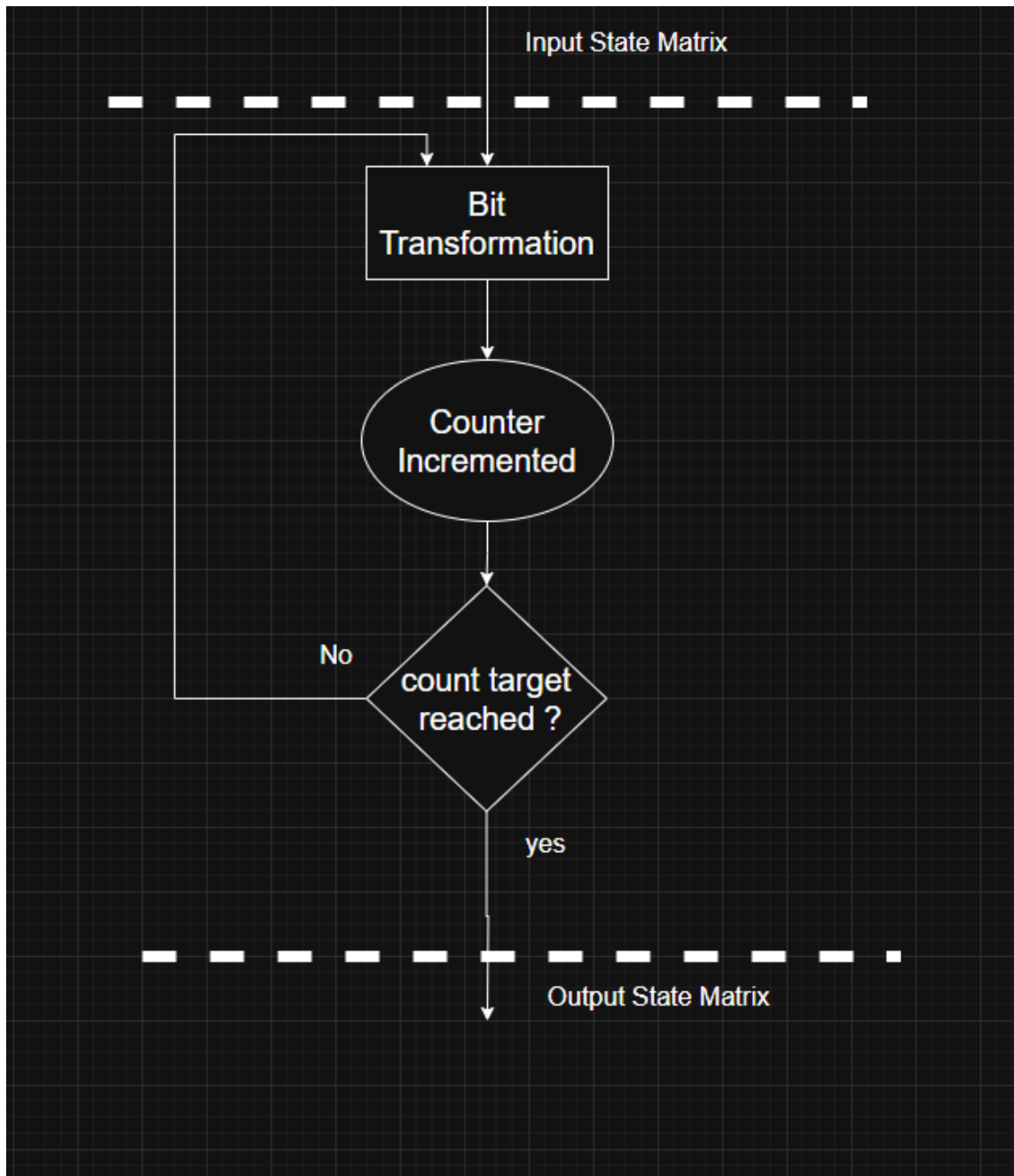


Figure 12, showcases a brief example of design approach

The second approach, which included my attempt to accelerate the algorithm, included a handful of alterations to functions as they were described in [17], in this implementation I favoured using large register greatly reducing the number of clocks cycles the functions took as well as simplifying some of the logic involved by using shifting operations rather than calculating each individual bit's movement. Additionally, I altered the way the state matrix was stored opting to assume all 64 bits of width were used regardless of the actual width of the bitstring. This means that instead of only occupying the area of the state matrix that is needed the bitstring would occupy the whole matrix regardless of size, for example if my bitstring is 50 bits long instead of only occupying the first 50 elements of the matrix I am now occupying the whole matrix regardless of the length of the bitstring meaning that if I have a 50 bit long bitstring I occupy the first 2 element and then populate the rest of the lane with zeros. This means that the position of index's is always the same regardless of the length of the bit string.

3.4 Top Level File

As the functions themselves were in individual modules a header file was needed to control said functions both implementations use the exact same header file with a minor difference due to the second implementation using a different way to index the state matrix. As can be seen in diagram 13, the top-level file is just a state machine that waits for the flags from the respective modules to be sent before it activates the next function, the output state matrixes fed into the next modules, reading the state as they are activated. The state machine also has a buffer state where it waits to be toggled.

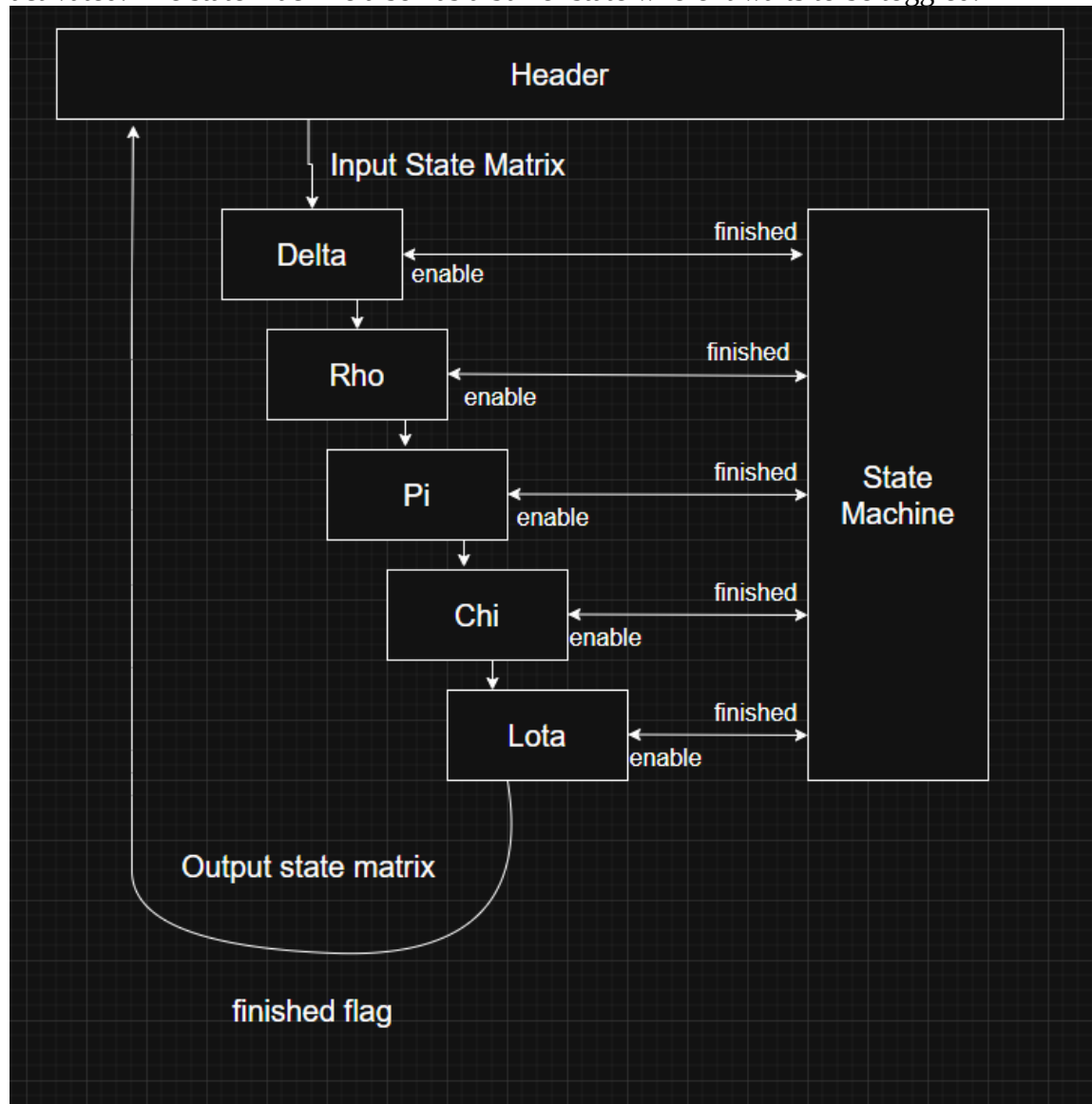


Figure 13 provides an overview of the structure of the header file.

3.5 Delta

3.5.1 Implementation 1

As the delta function affects all three axes, I created 3 counters, for each respective axis the counters were staggered so that when one counter met its target it resulted in the next counter incrementing and the counter that met its target resetting. This process is repeated until all the counters have reached their target at the same time, the values of

the current count of each counter is being used to keep track of the current index that the function is on. This process effectively represents a traversal through the state matrix one bit at a time, each bit's rearrangement is calculated in one cycle using the pseudocode that is present in 2.7.1. The Module itself is split into three stages as follows; stage 1: XOR all the columns (calculating array, C), stage 2 : using a series of XOR operations to calculate the array D, stage 3: XOR the input state matrix with the array D to get the output state Matrix.

3.5.2 Implementation 2

This implementation abused the fact that all the elements were in the same plane and that values that are subject to the XOR operation together are always in the same relative position to each other this is a direct result of choosing to use a different method for state matrix indexing. As a result, instead of computing the position of the bits that are XOR'ed together I created three copies of the state matrix and shifted two of them so that all the elements affected by XOR together are in the same index in their respective matrix's meaning I just need to XOR the matrices together. The shifted values that are deleted when shifted out of the range of the lane are saved before the shift so that they can be appended to the correct end of the matrix.

3.6 Rho

3.6.1 Implementation 1

Similar to the implementation of the delta function I used a counter to keep track of the z coordinate of the affected bit as well as the offset. I used the equation specified in [17] to find the offsets of the bits depending on their x and y coordinate and then using the current count the lanes were offset.

3.6.2 Implementation 2

As the values of the offsets for each lane are fixed, instead of calculating them at run time, I created an array that stored all the precalculated values, seeing as the only variance between different executions is the length of the bitstring. All that's needed after creating the array is to bind the shift to the length of the bit string using modulus and then shifting the lanes by the returned value. The same feature is retained from the Delta function where shifted values are made circular around the matrix by saving the value.

3.7 Pi

3.7.1 Implementation 1

This implementation like the others uses a counter to keep track of the current index being rotated, the values to be rotated to are calculated at run time, however the calculation of the new index as well as the rotation of the bit itself are both performed in the same clock cycle.

3.7.2 Implementation 2

Pi algorithm rotates the index in a predetermined way there is zero fluctuation based on iteration count or length of the bitstring as a result the lanes in the state matrix can be assigned to each other unconditionally this is present in the code as a block of addresses

being assigned to each. This would be synthesized simply as a connection between the input and output matrix, this can be seen more clearly by Figure 14 where the rotations are clearly defined.

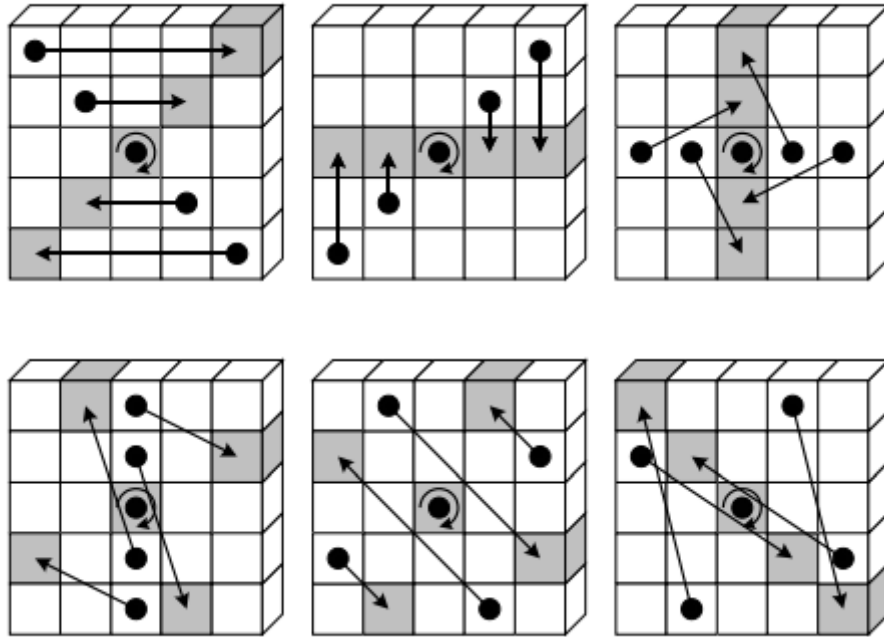


Figure 14, showcasing the rotation of the various lanes of the state matrix.

3.8 Chi

3.8.1 Implementation 1

The implementation is similar to the other implementations it uses a counter and calculates the new positions using the formula mentioned in [17].

3.8.2 Implementation 2

I implemented Chi extremely similarly to Delta I used the fact that elements are always relatively the same distance from each other in this version of the state matrix. I created four separate state matrix's all the size of a plane and shifted the elements such that the elements that meant to be combined in the combinational block are indexed to the same location remembering to save the bits that are lost when shifting to make the matrix circular.

Then once the arrays were shifted I simple computed the Boolean expression showcased in Figure 7.

3.9 Lota

Both implementations of Lota are similar with a slight alteration for the second implementation to incorporate the simplified rc generated mentioned in [31]. Both implementations use a counter in a similar to how they are used in the other algorithms in the first implementation, the counter is used to calculate the rc values in real time, all possible rc values are calculated and stored inside the RC array this essentially means that after the first iteration as long as the max iteration count is know the rc values don't

need to be calculated again. Once the values are calculated the ($x = 0$, $y = 0$) lane is modified by the RC value array using the XOR operation. The second implementation instead being affected by the whole RC array is only affected by the critical bits.

4 Testing and Evaluation

4.1 Testing Functionality

A set of test vectors was generated by the c program for every algorithm in the system. The size of the bitstring used was kept small to insure that any errors in either the test vector results or the simulated hardware results could be tracked by manually computing the values and then correcting the errors in the code. A consequence of deciding to test like this was that every algorithm in the second implementation had to redeclare the state matrix as inputting 1600-bit string into a testbench would be too cumbersome, this resulted in there being two copies of the second implementation one with each algorithm having its own state matrix transformation and the other without.

The bitstrings I used to test for functionality are as follows.
Where all bits are in little endian.

- Delta: 50'b0111011011011101010110010010010101111110000010011
- Rho: 50'b10101100011010111111010010001110000101011101111001
- Pi: 25'b1001010000001001101101000
- Chi: 25'b1001010000001001101110110
- Lota: 25'b0011010100001011100100010

The expected output bitstrings are as follow.

- Delta: 50'b10101100011010111111010010001110000101011101111001
- Rho: 50'b1010110001100111111100001001110000101011101110101
- Pi: 25'b1001000010010000001110110
- Chi: 25'b1011010100001011100100010
- Lota: 25'b0011010101001011100101010

As the outputs for both hardware implementations are equal to the expected values the functionality test is successful, see Figure 15 – 23, for waveforms of the output array as well as the period of the algorithms, due to the nature of the state matrix the waveform only shows the first 2 elements as the values are in their true indexes.

4.1.1 Test Bench Construction

The test benches used for both implementations were identical intentionally, this made it easier to test for errors as not only did I have the test vectors from the c program, but I also had a hardware implementation so if there was timing error with one of the implementation I could find out what specific clock cycle the error was originating from by comparing it to the other hardware implementation.

The testbench's themselves were light weight due to how modular the algorithms themselves were, and all constructed in a similar way. The algorithms were all tested against a clock signal with a period of 20ns, first the test bench would reset the algorithm by setting the reset input low. The only difference between each of the testbenches was the bit string that was input into the algorithm although tested separately I decided to feed the output from a successful test into the input of the next algorithm to verify the full algorithm itself was outputting the correct result.

4.2 Evaluation of Clock Cycles

Algorithm	First Implementation	Second Implementation
Delta	2140 ns – 107 CC	80ns – 4 CC
Rho	1000ns – 50 CC	40ns – 2 CC
Pi	600ns – 30 CC	40ns – 2 CC
Chi	600ns – 30 CC	60ns – 3 CC
Lota	1700ns – 85 CC	1700ns – 85 CC

Table 1, showcases the difference in algorithm period between the two implementations

When comparing the two implementations it's clear that the second implementation was optimised to reduce the amount of clock cycles required, the first implementation requires a total of 302 clock cycles to complete a full cycle, whereas the second implementation only requires 96 the majority of which come from Lota.

Assuming a respectable clock speed of 100MHz, and that both implementations can achieve it the second implementation can complete a cycle in 96 ns whereas the first implementation takes 302 this is a more than 3x increase in speed. Obviously, this isn't fully accurate as we had to assume the clock speed.

However, when we consider the algorithms that will have the highest amount of propagation delay, both Pi and Rho can be ignored as they are simply represented as connections between matrix indexes. Both implementations split Delta up into three stages therefore there is only one logic gate of propagation delay. On the contrary, the second implementation uses a massive shift register which depending on the internal clock could result in huge delays. Lastly there's Lota and Chi, Lota has one XOR gate of propagation whereas Chi has multiple and requires shifts in the second implementation and as such for this specific implementation I believe Chi has the largest impact on propagation delay.

Considering, Chi most likely has the largest impact on performance and its path consists of 1 shift and 3 logic gates I think its safe to assume that 100MHz is achievable.

4.3 Evaluation of Approach

The first implementation purposely was meant to resemble the description of the algorithm in the standard [17], the approach didn't try to utilise any of the advantages of a hardware approach and as a result it was clunky and required a lot of clock cycles to run. One of the biggest problems it faces is the fact that a lot of the calculations need to be done at run time as opposed to the second implementation where certain calculations can be avoided by abusing the fact that calculations are repeated over many cycles, so it is beneficial to simply compute them once and store them in a look up table. While this will increase the area of the implementation it will also drastically increase the speed of the implementation as demonstrated by the clock cycle improvements in the second implementation.

On the other hand, while the second implementation has a substantial clock cycle advantage, it pays for said advantage with larger registers as the first implementation is moving 1 bit at a time the register can be a lot smaller than the 64 bit shift registers that

4.4.2 Rho



Figure 17, waveform showcasing the output array of the first rho implementation as well as the finished flag

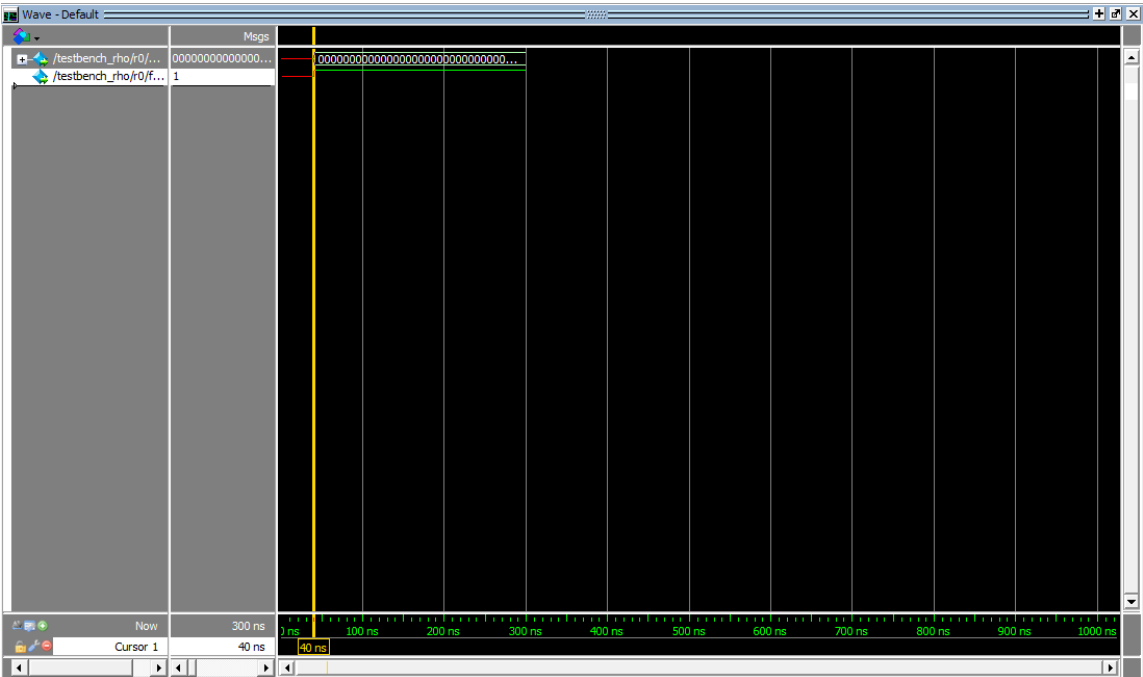


Figure 18, waveform showcasing the output array of the second rho implementation as well as the finished flag

4.4.3 Pi

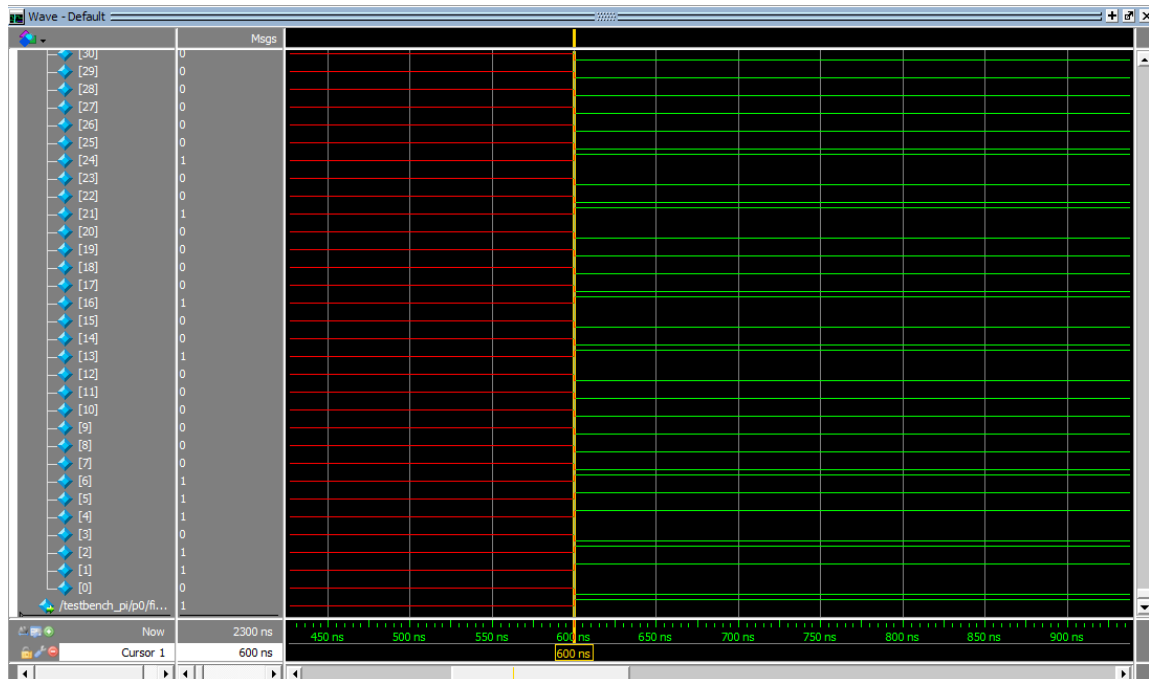


Figure 19, waveform showcasing the output array of the first pi implementation as well as the finished flag

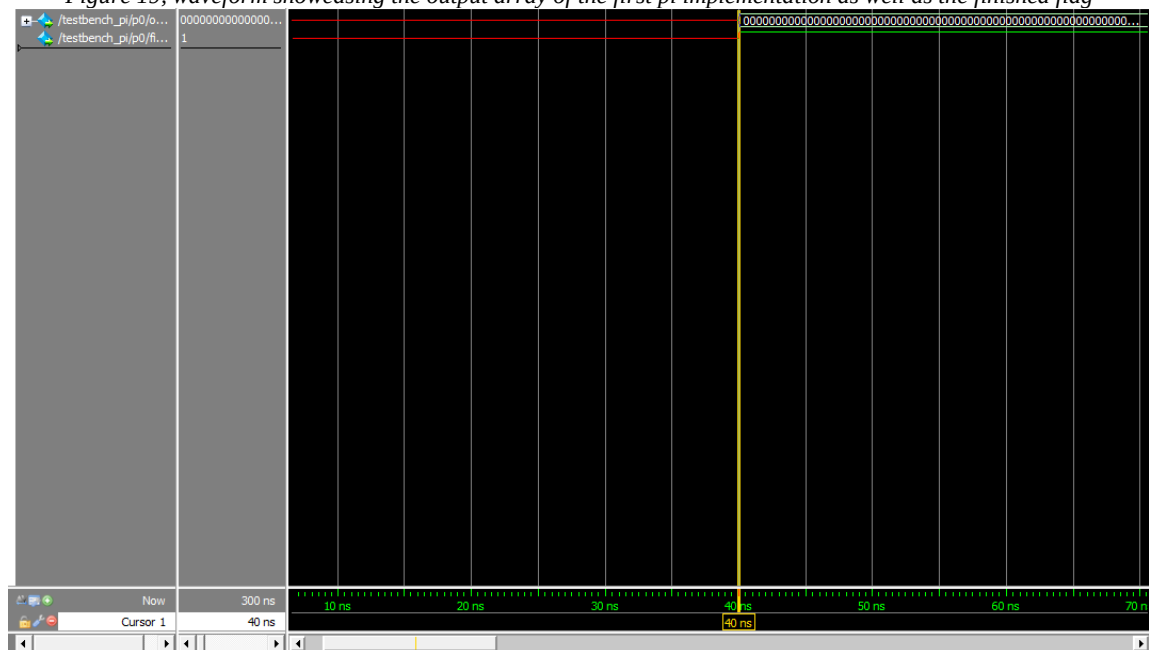


Figure 20, waveform showcasing the output array of the second pi implementation as well as the finished flag

4.4.4 Chi

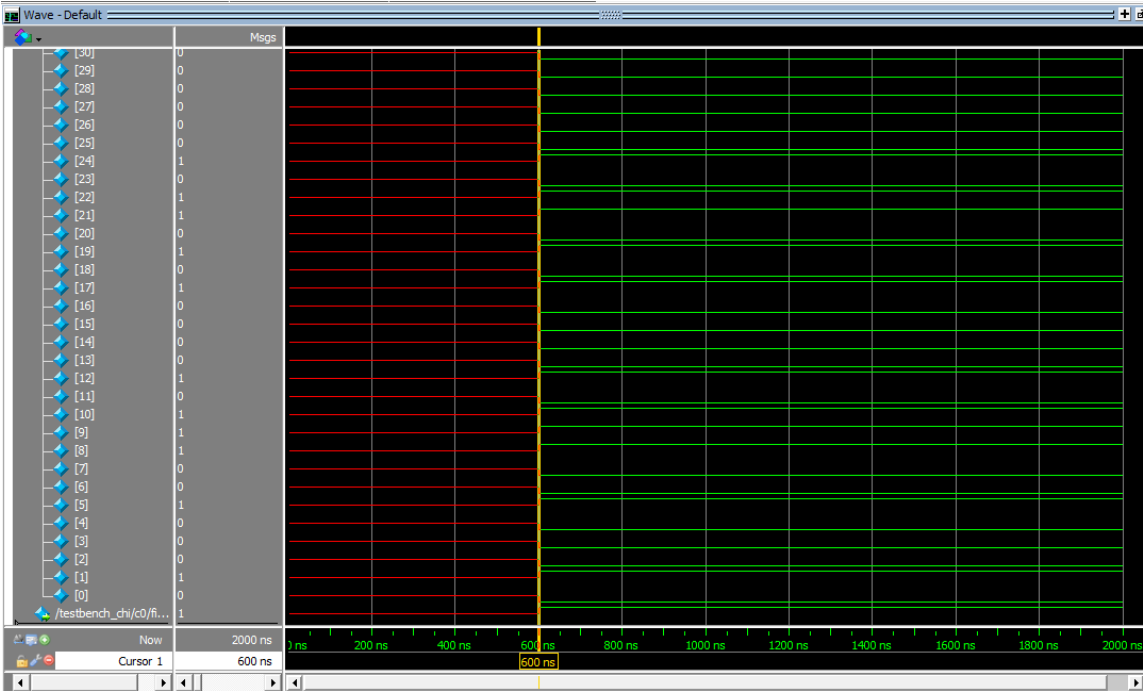


Figure 21, waveform showcasing the output array of the first chi implementation as well as the finished flag

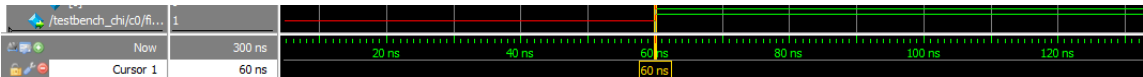


Figure 22, waveform showcasing the output array of the second chi implementation as well as the finished flag

4.4.5 Lota

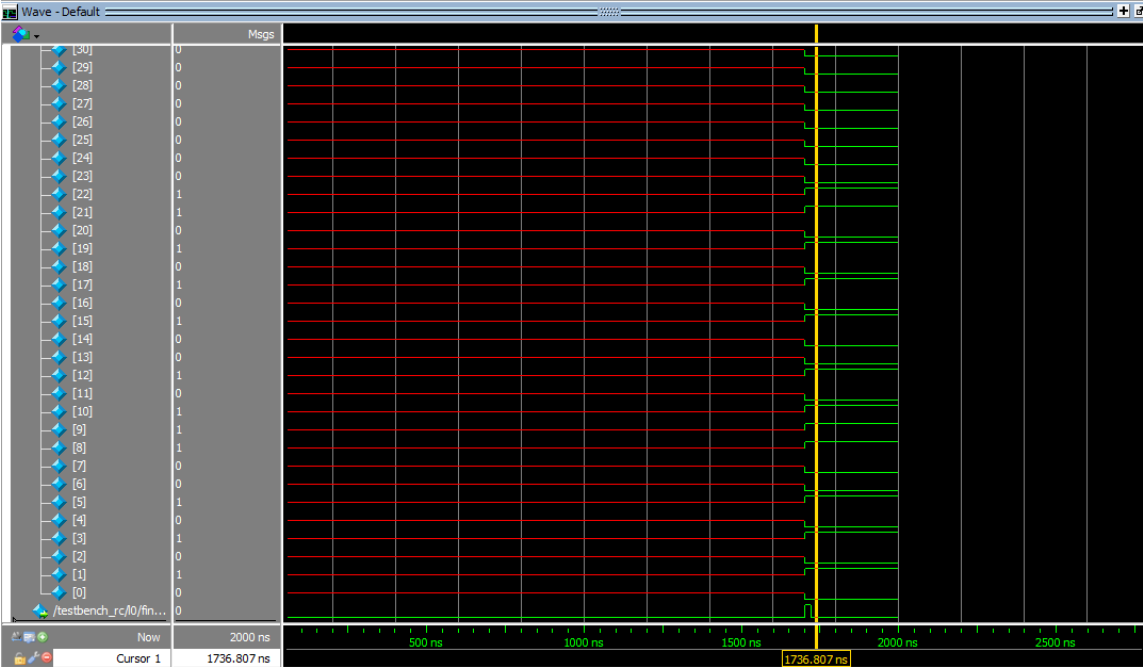


Figure 23, waveform showcasing the output array of the lota algorithm as well as the finished flag

5 Conclusions

This project aimed to create a hardware implementation of algorithms that are vital to FIPS 203, and then accelerate them, as a result this goal can be considered partially complete. The project resulted in the creation multiple functional system Verilog implementations of SHA-3 which were simulated using ModelSim to prove functionality, however it did not fully implement SHA-3 only the core and as such this goal can't be considered fully successful.

A secondary goal of the project was to create a resource that was more approachable then the already published literature. By providing two explained implementations as well as a rudimentary explanation of both cryptosystems and SHA-3's place in cryptography, this project could potentially allow someone to more easily approach the subject.

5.1 Project Management

The original plan as highlighted by the Gantt chart in Figure 24, was to develop a variety of different algorithms that feature in FIPS 203 with the goal of potentially having a fully functional cryptosystem. These included the NTT as well as RBG development however as more time passed, I ended up spending more time on tasks then I originally thought I would be resulting in me falling behind schedule.

Additionally, as can be seen in Figure 25 the goals of the project changed with time rather than simply developing one implementation of SHA-3 an additional implementation was developed and that implementation was accelerated, this resulted in more time needing to be spent working on the two SHA-3 cores. As a result of the changes in project goals as well as delays in development, previously planned milestones were abandoned. This resulted in some time on other milestones like RBG being wasted which could've been spent further improving my implementation of SHA-3.

5.2 Future work

There are many different directions a continuation of this project could go one of these directions would be the implementation of the full SHA-3 algorithm including the sponge architecture, this would allow for the implementation of techniques discussed in section 2.8.2. This would include unrolling the algorithm as described in [31] and a more in-depth investigation could be conducted on the relative performance of the method when compared to a traditional unrolled method. Additionally, unrolling the algorithm further could be investigated analysing whether the trade off for higher speed is even reasonable.

Another direction the project could go would be in the development of a random bit generator as described in [33], a hashing algorithm can be used to distribute the entropy of an entropy source creating the basis for a random bit generator. As a RBG is one of the fundamental Algorithms of cryptosystem work on it could potentially lead to the development of the rest of the system.

5.2.1 Original Gantt Chart

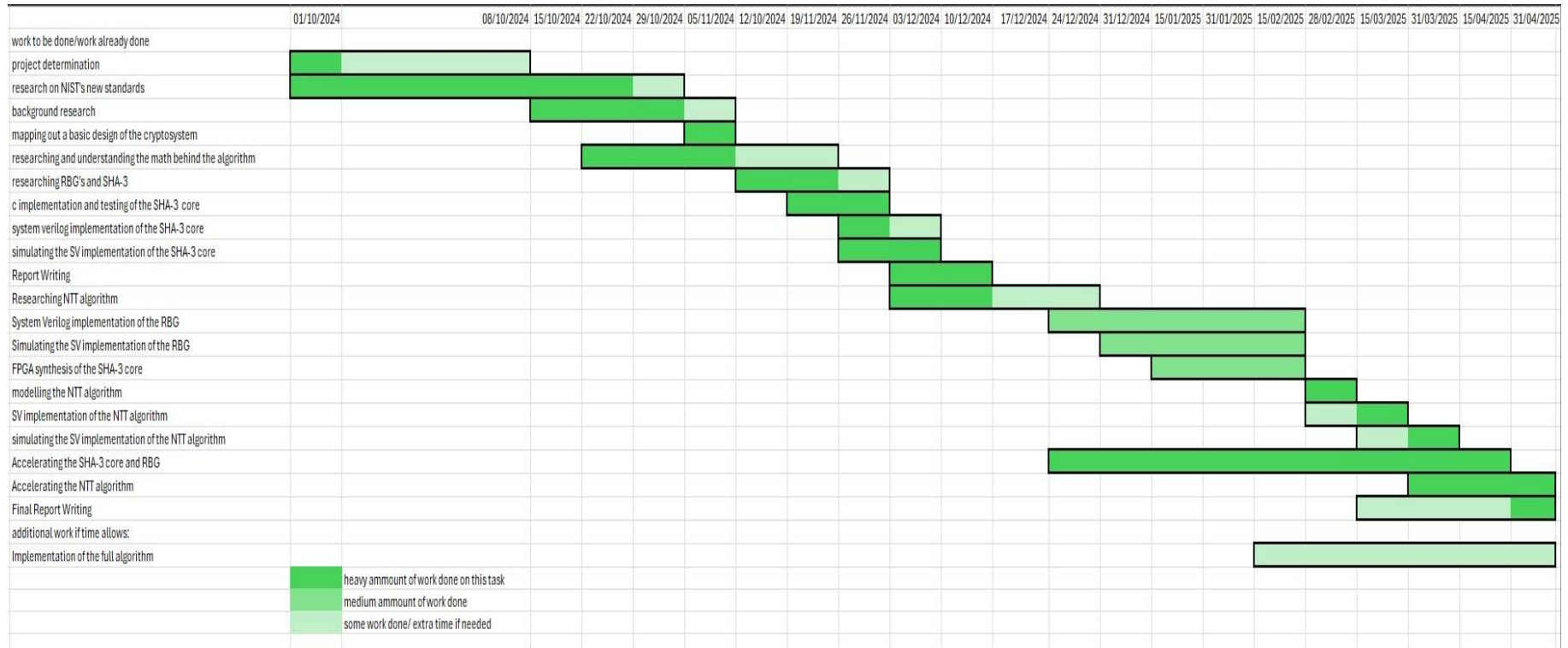


Figure 24, Gantt chart showing the plan made near the beginning of the project

5.2.2 Final Gantt Chart

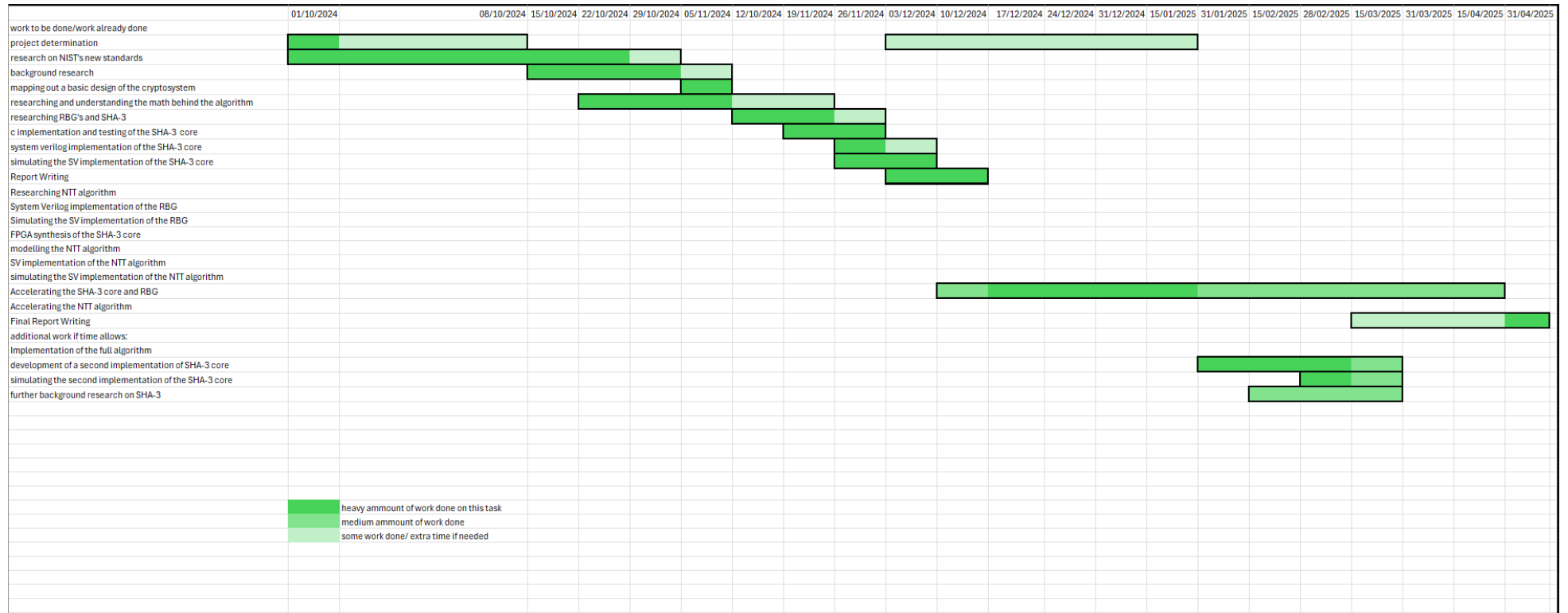


Figure 25, showcases the actual Gantt chart/ timeline

References

- [1] - J. R. Vacca, *Computer and information security handbook*. Waltham, Ma, Usa: Morgan Kaufmann Publishers, 2013.
- [2] - [Attitudes towards the internet in the UK 2024 | Statista](#)
- [3] - [UK cost of data breach 2024 | Statista](#)
- [4] - M. A. Shafique, A. Munir and I. Latif, "Quantum Computing: Circuits, Algorithms, and Applications," in *IEEE Access*, vol. 12, pp. 22296-22314, 2024, doi: 10.1109/ACCESS.2024.3362955.
- [5] - G. Kalai, "The argument against quantum computers," in *Quantum, Probability, Logic: The Work and Influence of Itamar Pitowsky*. Springer, 2020, pp. 399–422.
- [6] - "Quantum Computers and Algorithms: A Threat to Classical Cryptographic Systems," *International journal of engineering and advanced technology*, vol. 12, no. 5, pp. 25–38, Jun. 2023, doi: <https://doi.org/10.35940/ijeat.e4153.0612523>.
- [7] - M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge Cambridge University Press, 2010.
- [8] - NIST, "Post-Quantum Cryptography | CSRC | CSRC," CSRC | NIST, Jan. 03, 2017. <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [9] - S. and, "Post-Quantum Cryptography FIPS Approved | CSRC," *Nist.gov*, 2024. <https://csrc.nist.gov/News/2024/postquantum-cryptography-fips-approved>
- [10] - Gaithersburg MD NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," Jan. 2024, doi: <https://doi.org/10.6028/nist.fips.203>.
- [11] - O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing - STOC '05*, 2005, doi: <https://doi.org/10.1145/1060590.1060603>.
- [12] - K. Pietrzak, "Cryptography from Learning Parity with Noise," *Lecture notes in computer science*, pp. 99–114, Jan. 2012, doi: https://doi.org/10.1007/978-3-642-276606_9.
- [13] - luke, "Learning Parity with Noise (LPN) - zkPass - Medium," *Medium*, Mar. 09, 2023. <https://medium.com/zkpass/learning-parity-with-noise-lpn-55450fd4969c> (accessed Dec. 08, 2024).
- [14] - E. B. Barker and J. M. Kelsey, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," Jun. 2015, doi: <https://doi.org/10.6028/nist.sp.800-90ar1>.
- [15] - M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, "Recommendation for the entropy sources used for random bit generation," *NIST Special Publication 800-90B*, Jan. 2018, doi: <https://doi.org/10.6028/nist.sp.800-90b>.
- [16] - E. Barker, "Recommendation for Random Bit Generator (RBG) Constructions," 2024, doi: <https://doi.org/10.6028/nist.sp.800-90c.4pd>.
- [17] - M. J. Dworkin, "SHA-3 Standard: Permutation-Based Hash and ExtendableOutput Functions," Jul. 2015, doi: <https://doi.org/10.6028/nist.fips.202>.
- [18] - R. M. Gray, *Entropy and Information Theory*. Springer Science & Business Media, 2013.
- [19] - A. Cherkaoui, V. Fischer, A. Aubert and L. Fesquet, "Comparison of Self-Timed

Ring and Inverter Ring Oscillators as entropy sources in FPGAs," *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Dresden, Germany, 2012, pp. 1325-1330, doi: 10.1109/DATE.2012.6176697.

[20] – Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche and Ronny Van Keer
“Keccak Team,” *Keccak.team*, 2023. <https://keccak.team/hardware.html>

[21] - Nimbe, Peter & Frimpong, Samuel & Opoku, Michael. (2014). An Efficient Strategy for Collision Resolution in Hash Tables. *International Journal of Computer Applications*. 99. 35-41. 10.5120/17411-7990.

[22] - Edem, Swathi & Vivek, G. & Rani, G.. (2016). Role of Hash Function in Cryptography. 10-13. 10.22161/ijaers/si.3.

[23] - Bart Preneel, Katholieke Universiteit Leuven, K. Mercierlaan: Cryptographic Hash Functions

[24] - Khan, Burhan & Olanrewaju, Rashidah & Morshidi, Malik & Mir, Roohie & Kiah, Miss & Khan, Abdul. (2022). EVOLUTION AND ANALYSIS OF SECURED HASH ALGORITHM (SHA) FAMILY. *Malaysian Journal of Computer Science*. 35. 179-200. 10.22452/mjcs.vol35no3.1.

[25] - Kioon, Mary & Wang, ZhaoShun & Das, Shubra. (2013). Security Analysis of MD5 Algorithm in Password Storage. *Applied Mechanics and Materials*. 347-350. 10.2991/isccca.2013.177.

[26] - “Federal Information Processing Standards Publication: secure hash standard,” 1995, doi: <https://doi.org/10.6028/nist.fips.180-1>.

[27] - NIST, “NIST Retires SHA-1 Cryptographic Algorithm,” *NIST*, Dec. 2022, Available: <https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm>

[28] - S. and, “Secure Hash Standard (SHS),” *Nist.gov*, Feb. 25, 2004.
<https://csrc.nist.gov/pubs/fips/180-2/upd1/final> (accessed Apr. 28, 2025).

[29] - Sharma, Swati and ., Loveneet and Khanum, Saba, Performance analysis of SHA 2 and SHA 3 (March 28, 2022). *Proceedings of the International Conference on Innovative Computing & Communication (ICICC) 2022*, Available at SSRN: <https://ssrn.com/abstract=4068861> or <http://dx.doi.org/10.2139/ssrn.4068861>

[30] - “Keccak Team,” *keccak.team*. https://keccak.team/keccak_specs_summary.html

[31] - Sideris, Argyrios & Sanida, Theodora & Dasygenis, Minas. (2023). Hardware acceleration design of the SHA-3 for high throughput and low area on FPGA. *Journal of Cryptographic Engineering*. 14. 1-13. 10.1007/s13389-023-00334-0.

- [32] - Assad, Fatima & Fettach, Mohamed & el Otmani, Fadwa & Tragha, Abderrahim. (2022). High-performance FPGA implementation of the secure hash algorithm 3 for single and multi-message processing. International Journal of Electrical and Computer Engineering (IJECE). 12. 1324. 10.11591/ijece.v12i2.pp1324-1333.
- [33] - E. B. Barker and J. M. Kelsey, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators," Jun. 2015, doi: <https://doi.org/10.6028/nist.sp.800-90ar1>.

Appendix A. C code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

void populateSArr(int width, int* arr);
int* hashing(int* sArr, int length);
void delta(int* arr, int width);
int mod(int a, int b);
void rho(int* arr, int width);
void pi(int* arr, int width);
void chi(int* arr, int width);
int rc(int t);
void lota(int* arr, int width, int index);
int main(){
    int width = 1;
    int arr[(width*25)];
    int temp;
    populateSArr(width, arr);
    for(int i =0;i<(width*25);i++){
        //printf("array index %d = %d\n",i,arr[i]);
    }
    int lengthOfArr = sizeof(arr)/sizeof(arr[1]);
    hashing(arr,lengthOfArr);
    delta(arr,width);
    for(int i =0;i<(width*25);i++){
        //printf("array index %d = %d\n",i,arr[i]);
    }
    rho(arr,width);
    pi(arr,width);
    chi(arr,width);
    for(int i =0;i<(width*25);i++){
        //printf("array index %d = %d\n",i,arr[i]);
    }
    /*lota(arr,width,1);
    for(int i =0;i<(width*25);i++){
        //printf("array index %d = %d\n",i,arr[i]);
    }
    */

    return 0;
}

int* hashing(int* sArr, int length){
    int width = length+1/25;
}

void populateSArr(int width, int* arr){
    for(int i =0;i<(width*25);i++){
        int r = rand() % 2;
        arr[i] = r;
    }
}

void delta(int* arr, int width){
    int cArray[(width*5)];
```

```

int dArray[ (width*5) ];
for(int x = 0;x<5;x++){
    for(int z = 0;z<width;z++){
        cArray[ (width*x)+z]=arr[width*(5*(0)+x)+z]^arr[width*(5*(1)
+x)+z]^arr[width*(5*(2)+x)+z]^arr[width*(5*(3)+x)+z]^arr[width*(5*(4)+x
)+z];
    }
}

for(int x;x<5;x++){
    for(int z = 0;z < width;z++){
        dArray[ (width*x)+z]=cArray[width*(mod((x-
1),5))+z]^cArray[width*(mod((x+1),5))+(mod((z-1),width))];
    }
}

for(int x=0;x<5;x++){
    for(int y=0;y<5;y++){
        for(int z=0;z<width;z++){
            arr[width*(5*(y)+x)+z]=arr[width*(5*(y)+x)+z]^dArray[ (w
idth*x)+z];
        }
    }
}

int lengthOfArr = sizeof(cArray)/sizeof(cArray[1]);
for(int i =0;i<(width*5);i++){
    //printf("carray index %d = %d\n",i,cArray[i]);
}

for(int i =0;i<(width*5);i++){
    //printf("darray index %d = %d\n",i,dArray[i]);
}
for(int i =0;i<(width*25);i++){
    //printf("array index %d = %d\n",i,arr[i]);
}
}

void rho(int* arr,int width){
    int x = 1;
    int y = 0;
    int temp = 0;
    int arrT[ (width*25) ];
    memcpy(arrT,arr,width*25);
    int tempZ;
    for(int i =0;i<(width*25);i++){
        //printf("array index %d = %d\n",i,arr[i]);
    }

    for(int z =0;z<width;z++){
        for(int t =0;t<24;t++){
            tempZ = z-(t+1)*(t+2)*0.5;
            arrT[width*(5*(y)+x)+z]=arr[width*(5*(y)+x)+mod(tempZ,width
)];
            temp = y;
            y = mod(((2*x)+(3*y)),5);
            x=temp;
            //printf(" tempz = %d\n",tempZ);
            //printf(" x = %d y = %d\n",x,y);

        }
    }
    memcpy(arr,arrT,width*25);
}

```



```

        for(int i =0;i<(width*25);i++){
            //printf("array index %d = %d\n",i,arr[i]);
        }
    }
    void pi(int* arr,int width){
        for(int i =0;i<(width*25);i++){
            //printf("array index %d = %d\n",i,arr[i]);
        }
        int arrT[(width*25)];
        memcpy(arrT,arr,width*25);
        for(int y = 0;y<5;y++){
            for(int x = 0;x<5;x++){
                for(int z = 0;z<width;z++){
                    arrT[width*(5*(y)+x)+z]=arr[width*(5*(x)+(mod((x+(3*y)),5))+z)];
                }
            }
        }
        memcpy(arr,arrT,width*25);
        for(int i =0;i<(width*25);i++){
            //printf("array index %d = %d\n",i,arr[i]);
        }
    }
    void chi(int* arr,int width){
        for(int i =0;i<(width*25);i++){
            printf("array index %d = %d\n",i,arr[i]);
        }
        int arrT[(width*25)];
        memcpy(arrT,arr,width*25);
        for(int y = 0;y<5;y++){
            for(int x = 0;x<5;x++){
                for(int z =0;z<width;z++){
                    arrT[width*(5*(y)+x)+z]=arr[width*((5*y)+x)+z]^((arr[width*((5*y)+(mod((x+1),5))+z]^1)&arr[width*((5*y)+(mod((x+2),5))+z]]);
                }
            }
        }
        memcpy(arr,arrT,width*25);
        for(int i =0;i<(width*25);i++){
            printf("array index %d = %d\n",i,arr[i]);
        }
    }
    int rc(int t){
        int tempT = mod(t,255);
        if(tempT==0){
            return 1;
        }else{
            int rarr[9] = {1,0,0,0,0,0,0,0,0};
            for(int i = 0;i<tempT;i++){
                for(int j =9;j>0;j--){
                    rarr[j] = rarr[j-1];
                }

                rarr[0] = 0;
                rarr[0]=rarr[0]^rarr[8];
                rarr[4]=rarr[4]^rarr[8];
                rarr[5]=rarr[5]^rarr[8];
                rarr[6]=rarr[6]^rarr[8];
            }
            //for(int b = 0;b<1;b++){

```

```

        //printf("%d",rarr[b]);

    //}
    int b = 0;
    //printf("\n");
    //printf("%d\n",rarr[0]);
    return rarr[0];
}
}
void lota(int* arr,int width, int index){
    int rC[width];
    memset(rC,0,width*sizeof(rC[0]));
    int arrT[(width*25)];
    memcpy(arrT,arr,width*25);
    int lw = log2(width);
    for(int j = 0;j<=lw;j++){
        int result = pow(2,j);
        rC[(result)-1]=rc(j+(7*index));
        //for(int i =0;i<(width);i++){
            //printf("%d %d\n",j+(7*index),rc(j+(7*index)));

            //printf("array index %d = %d\n",i,rC[i]);

        //}
    }
    for(int z = 0;z<width;z++){
        //printf("array index %d = %d\n",z,rC[z]);
        //printf("array index %d = %d\n",z,arrT[z]);
        arrT[z]=arr[z]^rC[z];
    }
    memcpy(arr,arrT,width*25);
}
int mod(int a, int b){
    int ret = (a%b +b)%b;
    return ret;
}

```

Appendix B. Code for implementation 1

Delta

```
module delta(input logic [1599:0] arr, input logic [6:0] width, input logic
clk, n_rst, enable, output logic [1599:0] outArr, output logic finished);
```

```
enum {waiting, stage1, instage2, stage2, instage3, stage3} pState, nState;
```

```
logic
inc_cx, cx_FLG, inc_cy, cy_FLG, inc_cz, cz_FLG, cx_inFLG, cy_inFLG, cz_inFLG;
logic [6:0] count_z, count_x, count_y, cur_x, cur_y, cur_z;
```

```
counter
cx(.clk(clk), .n_rst(n_rst), .inc(inc_cx), .count(count_x), .rch_FLG(cx_FLG), .cur_index(cu
r_x), .in_FLG(cx_inFLG)); // counters for each axis
counter
cy(.clk(clk), .n_rst(n_rst), .inc(inc_cy), .count(count_y), .rch_FLG(cy_FLG), .cur_index(cu
r_y), .in_FLG(cy_inFLG));
counter
cz(.clk(clk), .n_rst(n_rst), .inc(inc_cz), .count(count_z), .rch_FLG(cz_FLG), .cur_index(cur
_z), .in_FLG(cz_inFLG));
```

```
logic [(64*5)-1:0] cArr, dArr;
```

```
always_ff@(posedge clk, negedge n_rst)
```

```
begin
if(!n_rst)
begin
```

```
end
else
begin
```

```
pState <= nState;
```

```
end
end
```

```
always_comb
```

```
begin
inc_cx = 1'b0; // init flags
inc_cy = 1'b0;
```

```

inc_cz = 1'b0;
cx_inFLG = 1'b1;
cy_inFLG = 1'b1;
cz_inFLG = 1'b1;

unique case(pState)
  waiting:begin
    count_x = '0;
    count_y = '0;
    count_z = '0;
    if(enable)
      begin
        nState = stage1;
        count_x = 6'b000100; //init counter target
        count_y = 6'b000100;
        count_z = width-1;
      end
    end

  end

  stage1:begin // calculating array C
    if((cur_x == count_x) && (cur_z == count_z))
      begin
        nState = instage2;
        cz_inFLG = 1'b0;
        cx_inFLG = 1'b0;
      end
    if(cur_z == count_z)
      begin
        inc_cx = 1'b1;
        cz_inFLG = 1'b0;

      end
    else
      begin
        inc_cz = 1'b1;
      end
    cArr[(width*cur_x)+cur_z]=arr[width*(5*(0)+cur_x)+cur_z]^arr[width*(5*(1
)+cur_x)+cur_z]^arr[width*(5*(2)+cur_x)+cur_z]^arr[width*(5*(3)+cur_x)+cur_z]^arr[
width*(5*(4)+cur_x)+cur_z];

  end

  instage2:begin //giving a cycle for the clock to reset
    nState = stage2;
  end

  stage2:begin //calculating array d
    if((cur_x == count_x) && (cur_z == count_z))

```

```

begin
    nState = instage3;
    cz_inFLG = 1'b0;
    cx_inFLG = 1'b0;
end
if(cur_z == count_z)
begin
    inc_cx = 1'b1;
    cz_inFLG = 1'b0;
end
else
begin
    inc_cz = 1'b1;
end
end
if((cur_x == 0)&&(cur_z==0))
    dArr[(width*cur_x)+cur_z]=cArr[width*(7'b0000100)+cur_z]^cArr[width*
((cur_x+1)%5)+(width-1)];
else if(cur_x == 0)
begin
    dArr[(width*cur_x)+cur_z]=cArr[width*(7'b0000100)+cur_z]^cArr[width*
((cur_x+1)%5)+((cur_z-1)%width)];
end
else if(cur_z==0)
begin
    dArr[(width*cur_x)+cur_z]=cArr[width*((cur_x-
1)%5)+cur_z]^cArr[width*((cur_x+1)%5)+(width-1)];
end
else
    dArr[(width*cur_x)+cur_z]=cArr[width*((cur_x-
1)%5)+cur_z]^cArr[width*((cur_x+1)%5)+((cur_z-1)%width)];
end

instage3:begin //giving a cycle for the clock to reset
    nState = stage3;
end

stage3:begin //calculating the final state matrix
    if((cur_x == count_x) && (cur_z == count_z) && (cur_y == count_y))
begin
    nState = waiting;
    cz_inFLG = 1'b0;
    cx_inFLG = 1'b0;
    cy_inFLG = 1'b0;
    finished = 1'b1;
end
if((cur_x == count_x) && (cur_z == count_z))
begin
    cz_inFLG = 1'b0;
    cx_inFLG = 1'b0;
    inc_cy = 1'b1;

```

```

        end
        if((cur_z == count_z))
        begin
            cz_inFLG = 1'b0;
            inc_cx = 1'b1;
        end
        else
        begin
            inc_cz = 1'b1;
        end
        outArr[width*(5*(cur_y)+cur_x)+cur_z]=arr[width*(5*(cur_y)+cur_x)+cur_z
]^dArr[(width*cur_x)+cur_z];

    end

    default:nState = waiting;
endcase

end

endmodule

```

Rho

```

module chi (input logic [1599:0] arr, input logic [6:0] width,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);

    enum {waiting,stage1} pState,nState;

    logic
inc_cx,cx_FLG,inc_cy,cy_FLG,inc_cz,cz_FLG,cx_inFLG,cy_inFLG,cz_inFLG;
    logic [6:0] count_z,count_x,count_y,cur_x,cur_y,cur_z;

    logic [1599:0] tArr;

    counter
cx(.clk(clk),.n_rst(n_rst),.inc(inc_cx),.count(count_x),.rch_FLG(cx_FLG),.cur_index(cu
r_x),.in_FLG(cx_inFLG));
    counter
cy(.clk(clk),.n_rst(n_rst),.inc(inc_cy),.count(count_y),.rch_FLG(cy_FLG),.cur_index(cu
r_y),.in_FLG(cy_inFLG));

```

```

        counter
        cz(.clk(clk),.n_rst(n_rst),.inc(inc_cz),.count(count_z),.rch_FLG(cz_FLG),.cur_index(cur
        _z),.in_FLG(cz_inFLG));

```

```

always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
    begin
        end
    else
    begin
        pState<=nState;
    end
end

```

```

always_comb
begin
    inc_cz = 1'b0;
    inc_cy = 1'b0;
    inc_cx = 1'b0;
    cz_inFLG = 1'b1;
    cy_inFLG = 1'b1;
    cx_inFLG = 1'b1;

```

```

unique case(pState)

```

```

    waiting:begin
        count_x = '0;
        count_z = '0;
        count_y = '0;

        if(enable)
        begin
            nState = stage1;
            tArr = arr;
            count_x = 6'b000100;
            count_y = 6'b000100;
            count_z = width-1;

```

```

        end
    end

```

```

    stage1:begin
        if((cur_x == count_x) && (cur_z == count_z) && (cur_y == count_y))
        begin
            nState = waiting;
            cz_inFLG = 1'b0;
            cx_inFLG = 1'b0;
            cy_inFLG = 1'b0;

```

```

        finished = 1'b1;
        outArr = tArr;
    end
    if((cur_x == count_x) && (cur_z == count_z))
    begin
        cz_inFLG = 1'b0;
        cx_inFLG = 1'b0;
        inc_cy = 1'b1;

    end
    if((cur_z == count_z))
    begin
        cz_inFLG = 1'b0;
        inc_cx = 1'b1;
    end

    else
    begin
        inc_cz = 1'b1;
    end

    end

    tArr[width*(5*(cur_y)+cur_x)+cur_z]=arr[width*((5*cur_y)+cur_x)+cur_z]^(
(arr[width*((5*cur_y)+((cur_x+1)%5))+cur_z]^1'b1)&arr[width*((5*cur_y)+((cur_x+2)
%5))+cur_z]);

end

default:nState = waiting;

endcase
end

endmodule

```

Pi

```

module pi (input logic [1599:0] arr, input logic [6:0] width,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);

```



```

enum {waiting,stage1} pState,nState;

logic
inc_cx,cx_FLG,inc_cy,cy_FLG,inc_cz,cz_FLG,cx_inFLG,cy_inFLG,cz_inFLG;
logic [6:0] count_z,count_x,count_y,cur_x,cur_y,cur_z;

logic [1599:0] tArr;

counter
cx(.clk(clk),.n_rst(n_rst),.inc(inc_cx),.count(count_x),.rch_FLG(cx_FLG),.cur_index(cu
r_x),.in_FLG(cx_inFLG));
counter
cy(.clk(clk),.n_rst(n_rst),.inc(inc_cy),.count(count_y),.rch_FLG(cy_FLG),.cur_index(cu
r_y),.in_FLG(cy_inFLG));
counter
cz(.clk(clk),.n_rst(n_rst),.inc(inc_cz),.count(count_z),.rch_FLG(cz_FLG),.cur_index(cur
_z),.in_FLG(cz_inFLG));

always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
        begin
            end
        else
            begin
                pState<=nState;
            end
        end
end

always_comb
begin
    inc_cz = 1'b0;
    inc_cy = 1'b0;
    inc_cx = 1'b0;
    cz_inFLG = 1'b1;
    cy_inFLG = 1'b1;
    cx_inFLG = 1'b1;

    unique case(pState)

        waiting:begin
            count_x = '0;
            count_z = '0;
            count_y = '0;

            if(enable)
            begin
                nState = stage1;
                tArr = arr;
            end
        end
    endcase
end

```

```

        count_x = 6'b000100;
        count_y = 6'b000100;
        count_z = width-1;

    end
end

stage1:begin
    if((cur_x == count_x) && (cur_z == count_z) && (cur_y == count_y))
    begin
        nState = waiting;
        cz_inFLG = 1'b0;
        cx_inFLG = 1'b0;
        cy_inFLG = 1'b0;
        finished = 1'b1;
        outArr = tArr;
    end
    if((cur_x == count_x) && (cur_z == count_z))
    begin
        cz_inFLG = 1'b0;
        cx_inFLG = 1'b0;
        inc_cy = 1'b1;

    end
    if((cur_z == count_z))
    begin
        cz_inFLG = 1'b0;
        inc_cx = 1'b1;
    end

    else
    begin
        inc_cz = 1'b1;
    end

    end

    tArr[width*(5*(cur_y)+cur_x)+cur_z]=arr[width*(5*(cur_x)+((cur_x+(3*cur_
y))%5))+cur_z];

end

default:nState = waiting;

```

```

        endcase
    end

endmodule

```

Chi

```

module chi (input logic [1599:0] arr, input logic [6:0] width, input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);

    enum {waiting,stage1} pState,nState;

    logic
inc_cx,cx_FLG,inc_cy,cy_FLG,inc_cz,cz_FLG,cx_inFLG,cy_inFLG,cz_inFLG;
    logic [6:0] count_z,count_x,count_y,cur_x,cur_y,cur_z;

    logic [1599:0] tArr;

    counter
cx(.clk(clk),.n_rst(n_rst),.inc(inc_cx),.count(count_x),.rch_FLG(cx_FLG),.cur_index(cu
r_x),.in_FLG(cx_inFLG));
    counter
cy(.clk(clk),.n_rst(n_rst),.inc(inc_cy),.count(count_y),.rch_FLG(cy_FLG),.cur_index(cu
r_y),.in_FLG(cy_inFLG));
    counter
cz(.clk(clk),.n_rst(n_rst),.inc(inc_cz),.count(count_z),.rch_FLG(cz_FLG),.cur_index(cur
_z),.in_FLG(cz_inFLG));

    always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
begin
end
    else
begin
        pState<=nState;
    end
end

    always_comb
begin
    inc_cz = 1'b0;
    inc_cy = 1'b0;
    inc_cx = 1'b0;
    cz_inFLG = 1'b1;
    cy_inFLG = 1'b1;

```

```
cx_inFLG = 1'b1;
```

```
unique case(pState)
```

```
  waiting:begin
```

```
    count_x = '0;
```

```
    count_z = '0;
```

```
    count_y = '0;
```

```
    if(enable)
```

```
    begin
```

```
      nState = stage1;
```

```
      tArr = arr;
```

```
      count_x = 6'b000100;
```

```
      count_y = 6'b000100;
```

```
      count_z = width-1;
```

```
    end
```

```
  end
```

```
  stage1:begin
```

```
    if((cur_x == count_x) && (cur_z == count_z) && (cur_y == count_y))
```

```
    begin
```

```
      nState = waiting;
```

```
      cz_inFLG = 1'b0;
```

```
      cx_inFLG = 1'b0;
```

```
      cy_inFLG = 1'b0;
```

```
      finished = 1'b1;
```

```
      outArr = tArr;
```

```
    end
```

```
    if((cur_x == count_x) && (cur_z == count_z))
```

```
    begin
```

```
      cz_inFLG = 1'b0;
```

```
      cx_inFLG = 1'b0;
```

```
      inc_cy = 1'b1;
```

```
    end
```

```
    if((cur_z == count_z))
```

```
    begin
```

```
      cz_inFLG = 1'b0;
```

```
      inc_cx = 1'b1;
```

```
    end
```

```
  else
```

```
  begin
```

```
    inc_cz = 1'b1;
```

```
  end
```

```

        tArr[width*(5*(cur_y)+cur_x)+cur_z]=arr[width*((5*cur_y)+cur_x)+cur_z]^(
(arr[width*((5*cur_y)+((cur_x+1)%5))+cur_z]^1'b1)&arr[width*((5*cur_y)+((cur_x+2)
%5))+cur_z]);

```

```

    end

```

```

    default:nState = waiting;

```

```

    endcase

```

```

end

```

```

endmodule

```

Lota

```

module lota (input logic enable,input logic clk,n_rst,input logic [1599:0] arr,output logic
finished,output logic [1599:0] outArr);

```

```

    logic inc_ci,ci_FLG,inc_cj,cj_FLG,ci_inFLG,cj_inFLG;
    logic [7:0] count_i,count_j,cur_i,cur_j,tempT;

```

```

    logic [63:0] rC;
    logic [7:0] value;

```

```

    logic [1599:0] tArr;

```

```

    logic [8:0] rarr,temprr;
    counter
    ci(.clk(clk),.n_rst(n_rst),.inc(inc_ci),.count(count_i),.rch_FLG(ci_FLG),.cur_index(cur_i
),.in_FLG(ci_inFLG));

```

```

    enum { waiting,stage1,stage2} pState,nState;

```

```

    always_ff@(posedge clk,negedge n_rst)
    begin

```

```

    if(!n_rst)
    begin
    end
    else
    begin
        pState<=nState;
    end
end

always_comb
begin
    inc_ci = 1'b0;

    ci_inFLG = 1'b1;

    unique case(pState)
        waiting:begin
            count_i = '0;
            count_j = '0;
            finished = 1'b0;
            value = 8'b00000000;
            rC = '{64{1'b0}};

            if(enable)
            begin
                nState = stage1;

            end

        end

    end
    stage1:begin
        nState = stage2;
        rarr = 9'b000000001;
        tempT = (value + 8'b00000111) % 255;
        if(tempT == 0)
        begin

            rC[(2**value)-1] = 1'b1;
            value = value + 1'b1;
            nState = stage1;
        end
        count_i = (tempT)-1;
        if(value == 8'b00000111)
        begin
            nState = waiting;
            finished = 1'b1;
            outArr[0 +: 64] = arr[0 +: 64] ^ rC;
        end
    end
end

```

```

end
stage2:begin
    if(cur_i == count_i)
        begin

            rC[(2**value)-1] = rarr[0];
            ci_inFLG = 1'b0;
            value = value + 1'b1;
            nState = stage1;

        end
    else
        begin
            inc_ci = 1'b1;
        end
        rarr = rarr << 1;
        rarr[0]=rarr[0]^rarr[8];
        rarr[4]=rarr[4]^rarr[8];
        rarr[5]=rarr[5]^rarr[8];
        rarr[6]=rarr[6]^rarr[8];

    end

    default:nState = waiting;
endcase

end

endmodule

```

Appendix C. Code for Implementation 2

Delta

```
module delta(input logic [1599:0] arr, input logic [6:0] width,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);

    enum {waiting,init,stage1,stage2,stage3} pState,nState;

    logic enable_matrix;

    logic [1599:0] state_matrix;

    state_mat
m0(.clk(clk),.n_rst(n_rst),.enable(enable_matrix),.bitString(arr),.state_matrix(state_matri
x));

    logic [(64*5)-1:0] cArr,dArr,temp1_cArr,temp2_cArr,temp_cArr;

    always_ff@(posedge clk,negedge n_rst)
    begin
        if(!n_rst)
        begin

            end
            else
            begin

                pState <= nState;

            end
        end
    end

    always_comb
    begin

        unique case(pState)
        waiting:begin
            if(enable)
            begin
```



```

        enable_matrix = 1'b1;
        nState = init;

    end

end

init:begin
    nState = stage1;

end

stage1:begin
    cArr[319:0] = state_matrix[(0)*320 +: 320] ^ state_matrix[(1)*320 +: 320] ^
state_matrix[(2)*320 +: 320] ^ state_matrix[(3)*320 +: 320] ^ state_matrix[(4)*320 +:
320];
    nState = stage2;
end

stage2:begin
    temp_cArr = cArr >> 63;
    temp1_cArr = cArr >> 63;
    temp1_cArr[256 +: 63] = cArr[0 +: 63]; //array corresponding to C[(x+1) mod
5, (z-1) mod w]
    temp1_cArr[0] = cArr[64+(width-1)];
    temp1_cArr[64] = cArr[128+(width-1)];
    temp1_cArr[128] = cArr[192+(width-1)];
    temp1_cArr[192] = cArr[256+(width-1)];
    temp1_cArr[256] = cArr[width-1];
    temp2_cArr = cArr << 64;
    temp2_cArr[0 +: 64] = cArr[256 +: 64]; //array corresponding to C[(x-1) mod
5, z]
    dArr[319:0] = temp1_cArr[319:0] ^ temp2_cArr[319:0];
    nState = stage3;

end

stage3:begin
    outArr[(0)*320 +: 320] = state_matrix[(0)*320 +: 320] ^ dArr[319:0];
    outArr[(1)*320 +: 320] = state_matrix[(1)*320 +: 320] ^ dArr[319:0];
    outArr[(2)*320 +: 320] = state_matrix[(2)*320 +: 320] ^ dArr[319:0];
    outArr[(3)*320 +: 320] = state_matrix[(3)*320 +: 320] ^ dArr[319:0];
    outArr[(4)*320 +: 320] = state_matrix[(4)*320 +: 320] ^ dArr[319:0];

    finished = 1'b1;
    nState = waiting;
end

default:nState = waiting;
endcase

end

```

```
endmodule
```

Rho

```
module rho #(parameter [6:0] width = 7'b0000010)(input logic [1599:0] arr,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);
    enum {waiting,init,stage1} pState,nState;
    logic [5:0] n;
    logic [8:0] shift,shift1;
    logic [8:0] shift_values [24:0];
    logic [63:0] tempArr;
    logic enable_matrix;
    logic [1599:0] state_matrix;

    state_mat
m0(.clk(clk),.n_rst(n_rst),.enable(enable_matrix),.bitString(arr),.state_matrix(state_matri
x));

    always_ff@(posedge clk,negedge n_rst)
    begin
        if(!n_rst)
            begin
                end
            else
                begin
                    pState<=nState;
                end
            end
    end

    always_comb
    begin

        unique case(pState)

            waiting:begin
                if(enable)
                    begin

                        shift_values =
{9'd0,9'd1,9'd190,9'd28,9'd91,9'd36,9'd300,9'd6,9'd55,9'd276,9'd3,9'd10,9'd171,9'd153,9
'd231,9'd105,9'd45,9'd15,9'd21,9'd136,9'd210,9'd66,9'd253,9'd120,9'd78};
                        nState = init;
                        enable_matrix = 1'b1;

```

```

        end
    end

    init:begin

        nState=stage1;

    end

    stage1:begin
        for(n=0;n<25;n=n+1)
            begin
                shift[8:0] = shift_values[(24-n)];

                tempArr[0+:64] = state_matrix[n*64+:64] << shift[$clog2(width)-1:0];
                if(shift[$clog2(width)-1] > '0')begin
                    tempArr[0+:64] = tempArr[0+:64] | (tempArr[0+:64] >> width);
                end
                outArr[n*64+:64] = tempArr[0+:64];

            end
            finished = 1'b1;

            nState = waiting;
        end

        default:nState = waiting;

    endcase
end

endmodule

```

Pi

```

//store all z values in reg then move them to correct location
module pi(input logic [1599:0] arr, input logic [6:0] width,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);

```

```

    enum {waiting,init,stage1} pState,nState;

```

```

    logic enable_matrix;

```

```

logic [1599:0] state_matrix;

state_mat
m0(.clk(clk),.n_rst(n_rst),.enable(enable_matrix),.bitString(arr),.state_matrix(state_matri
x));

always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
    begin

    end
    else
    begin

        pState <= nState;

    end
end

always_comb
begin

    unique case(pState)
        waiting:begin
            if(enable)
            begin
                enable_matrix = 1'b1;
                nState = init;

            end
        end

        init:begin
            nState = stage1;

        end

        stage1:begin

            outArr[0 +: 64]  = state_matrix[0 +:64];
            outArr[64 +: 64] = state_matrix[384 +:64];
            outArr[128 +: 64] = state_matrix[768 +:64];
            outArr[192 +: 64] = state_matrix[1152 +:64];
            outArr[256 +: 64] = state_matrix[1536 +:64]; //ugly hardcoded block

```

```
        outArr[320 +: 64] = state_matrix[192 +:64]; //the alternative approach would
be to mess around with if statements and for loops
```

```
        outArr[384 +: 64] = state_matrix[576 +:64]; //as seen in pi.sv or you could
make a modulo computing block.
```

```
        outArr[448 +: 64] = state_matrix[640 +:64];
        outArr[512 +: 64] = state_matrix[1024 +:64];
        outArr[576 +: 64] = state_matrix[1408 +:64];
        outArr[640 +: 64] = state_matrix[64 +:64];
        outArr[704 +: 64] = state_matrix[448 +:64];
        outArr[768 +: 64] = state_matrix[832 +:64];
        outArr[832 +: 64] = state_matrix[1216 +:64];
        outArr[896 +: 64] = state_matrix[1280 +:64];
        outArr[960 +: 64] = state_matrix[256 +:64];
        outArr[1024 +: 64] = state_matrix[320 +:64];
        outArr[1088 +: 64] = state_matrix[704 +:64];
        outArr[1152 +: 64] = state_matrix[1088 +:64];
        outArr[1216 +: 64] = state_matrix[1472 +:64];
        outArr[1280 +: 64] = state_matrix[128 +:64];
        outArr[1344 +: 64] = state_matrix[512 +:64];
        outArr[1408 +: 64] = state_matrix[896 +:64];
        outArr[1472 +: 64] = state_matrix[960 +:64];
        outArr[1536 +: 64] = state_matrix[1344 +:64];
```

```
        finished = 1'b1;
        nState = waiting;
    end
```

```
        default:nState = waiting;
    endcase
```

```
end
```

```
endmodule
```

Chi

```
module chi#(parameter [6:0] width = 7'b00000001)(input logic [1599:0] arr,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);
```

```
    enum {waiting,init,stage1,stage2} pState,nState;
```

```

logic enable_matrix;

logic [1599:0] state_matrix;
logic [(64*5)-1:0] temp_not_y1, temp_2_y1, temp_not_y2, temp_2_y2, temp_not_y3,
temp_2_y3, temp_not_y4, temp_2_y4, temp_not_y5, temp_2_y5, one_xor;
logic [(64*5)-1:0] final_y1, final_y2, final_y3, final_y4, final_y5;

state_mat
m0(.clk(clk),.n_rst(n_rst),.enable(enable_matrix),.bitString(arr),.state_matrix(state_matri
x));

always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
        begin

        end
    else
        begin

            pState <= nState;

        end
    end

always_comb
begin

    unique case(pState)
        waiting:begin
            if(enable)
                begin
                    enable_matrix = 1'b1;
                    one_xor[(0)*64+:320] = {1'b1 + '0};
                    one_xor[(1)*64+:320] = {1'b1 + '0};
                    one_xor[(2)*64+:320] = {1'b1 + '0};
                    one_xor[(3)*64+:320] = {1'b1 + '0};
                    one_xor[(4)*64+:320] = {1'b1 + '0};
                    nState = init;

                end
            end

        init:begin

```

```

    nState = stage1;

end
stage1:begin

    temp_not_y1 = state_matrix[(0)*320 +: 320] >> 64;
    temp_not_y2 = state_matrix[(1)*320 +: 320] >> 64;
    temp_not_y3 = state_matrix[(2)*320 +: 320] >> 64;
    temp_not_y4 = state_matrix[(3)*320 +: 320] >> 64;
    temp_not_y5 = state_matrix[(4)*320 +: 320] >> 64;

    temp_2_y1 = state_matrix[(0)*320 +: 320] >> 128;
    temp_2_y2 = state_matrix[(1)*320 +: 320] >> 128;
    temp_2_y3 = state_matrix[(2)*320 +: 320] >> 128;
    temp_2_y4 = state_matrix[(3)*320 +: 320] >> 128;
    temp_2_y5 = state_matrix[(4)*320 +: 320] >> 128;

    temp_not_y1[320-64 +: 64] = state_matrix[(0)*320 +: 64];
    temp_not_y2[320-64 +: 64] = state_matrix[(1)*320 +: 64];
    temp_not_y3[320-64 +: 64] = state_matrix[(2)*320 +: 64];
    temp_not_y4[320-64 +: 64] = state_matrix[(3)*320 +: 64];
    temp_not_y5[320-64 +: 64] = state_matrix[(4)*320 +: 64];

    temp_2_y1[320-128 +: 128] = state_matrix[(0)*320 +: 128];
    temp_2_y2[320-128 +: 128] = state_matrix[(1)*320 +: 128];
    temp_2_y3[320-128 +: 128] = state_matrix[(2)*320 +: 128];
    temp_2_y4[320-128 +: 128] = state_matrix[(3)*320 +: 128];
    temp_2_y5[320-128 +: 128] = state_matrix[(4)*320 +: 128];

    nState = stage2;

end
stage2:begin

    temp_not_y1 = temp_not_y1 ^ one_xor;
    temp_not_y2 = temp_not_y2 ^ one_xor;
    temp_not_y3 = temp_not_y3 ^ one_xor;
    temp_not_y4 = temp_not_y4 ^ one_xor;
    temp_not_y5 = temp_not_y5 ^ one_xor;

    outArr[(0)*320 +:320] = state_matrix[(0)*320 +: 320] ^ (temp_not_y1[319:0]
& temp_2_y1[319:0]);
    outArr[(1)*320 +:320] = state_matrix[(1)*320 +: 320] ^ (temp_not_y2[319:0]
& temp_2_y2[319:0]);
    outArr[(2)*320 +:320] = state_matrix[(2)*320 +: 320] ^ (temp_not_y3[319:0]
& temp_2_y3[319:0]);
    outArr[(3)*320 +:320] = state_matrix[(3)*320 +: 320] ^ (temp_not_y4[319:0]
& temp_2_y4[319:0]);
    outArr[(4)*320 +:320] = state_matrix[(4)*320 +: 320] ^ (temp_not_y5[319:0]
& temp_2_y5[319:0]);

```

```

        finished = 1'b1;
        nState = waiting;
    end

```

```

        default:nState = waiting;
    endcase

```

```

end

```

```

endmodule

```

Lota

```

module lota (input logic enable,input logic clk,n_rst,input logic [1599:0] arr,output logic
finished,output logic [1599:0] outArr);

```

```

    logic inc_ci,ci_FLG,inc_cj,cj_FLG,ci_inFLG,cj_inFLG;
    logic [7:0] count_i,count_j,cur_i,cur_j,tempT;

```

```

    logic [63:0] rC;
    logic [7:0] value;

```

```

    logic [1599:0] tArr;

```

```

    logic [8:0] rarr,tempR;
    counter
    ci(.clk(clk),.n_rst(n_rst),.inc(inc_ci),.count(count_i),.rch_FLG(ci_FLG),.cur_index(cur_i
),.in_FLG(ci_inFLG));

```

```

    enum {waiting,stage1,stage2} pState,nState;

```



```

always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
        begin
        end
    else
        begin
            pState<=nState;
        end
    end

always_comb
begin
    inc_ci = 1'b0;

    ci_inFLG = 1'b1;

    unique case(pState)
        waiting:begin
            count_i = '0;
            count_j = '0;
            finished = 1'b0;
            value = 8'b00000000;
            rC = '{64{1'b0}};

            if(enable)
            begin
                nState = stage1;

            end

        end
        stage1:begin
            nState = stage2;
            rarr = 9'b000000001;
            tempT = (value + 8'b00000111) % 255;
            if(tempT == 0)
            begin

                rC[(2**value)-1] = 1'b1;
                value = value + 1'b1;
                nState = stage1;
            end
            count_i = (tempT)-1;
            if(value == 8'b00000111)
            begin
                nState = waiting;
                finished = 1'b1;
            end
        end
    endcase
end

```

```

        outArr[63] = arr[63] ^ rC[63];
        outArr[31] = arr[31] ^ rC[31];
        outArr[15] = arr[15] ^ rC[15];
        outArr[7] = arr[7] ^ rC[7];
        outArr[3] = arr[3] ^ rC[3];
        outArr[1] = arr[1] ^ rC[1];
        outArr[0] = arr[0] ^ rC[0];

        outArr[0+:64] = arr[0+:64] ^ rC;
    end

end
stage2:begin
    if(cur_i == count_i)
        begin

            rC[(2**value)-1] = rarr[0];
            ci_inFLG = 1'b0;
            value = value + 1'b1;
            nState = stage1;

        end
    else
        begin
            inc_ci = 1'b1;
        end
        rarr = rarr << 1;
        rarr[0]=rarr[0]^rarr[8];
        rarr[4]=rarr[4]^rarr[8];
        rarr[5]=rarr[5]^rarr[8];
        rarr[6]=rarr[6]^rarr[8];

    end

    default:nState = waiting;
endcase

end

endmodule

```

Appendix D. Header Files

Implementation 1

```
module sha3 #(parameter [6:0] width = 7'b0000010)(input logic [1599:0] arr,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);
```

```
    logic [1599:0] outputDelta, outputRho, outputPi, outputChi, outputLota, finalOutput;
    logic
enableDelta,enableRho,enablePi,enableChi,enableLota,finishedDelta,finishedRho,finish
edPi,finishedChi,finishedLota;
```

```
    enum {waiting,sDelta,sRho,sPi,sChi,sLota} pState,nState;
```

```
    delta
d0(.arr(arr),.outArr(outputDelta),.clk(clk),.n_rst(n_rst),.enable(enableDelta),.finished(fin
ishedDelta),.width(width));
    rho
r0(.arr(outputDelta),.outArr(outputRho),.clk(clk),.n_rst(n_rst),.enable(enableRho),.finish
ed(finishedRho),.width(width));
    pi p0(.arr(outputRho),.outArr(outputPi),.clk(clk),.n_rst(n_rst),.enable(enablePi),.finis
hed(finishedPi),.width(width));
    chi
c0(.arr(outputPi),.outArr(outputChi),.clk(clk),.n_rst(n_rst),.enable(enableChi),.finished(f
inishedChi),.width(width));
    lota
l0(.arr(outputChi),.outArr(outputLota),.clk(clk),.n_rst(n_rst),.enable(enableLota),.finishe
d(finishedLota));
```

```
    always_ff@(posedge clk,negedge n_rst)
```

```
    begin
        if(!n_rst)
            begin
```

```
                end
                else
                begin
```

```
                    pState <= nState;
```

```
                end
            end
```

```
    always_comb
    begin
```

unique case(pState)

```
waiting:begin
    enableDelta = 1'b0;
    enableRho = 1'b0;
    enablePi = 1'b0;
    enableChi = 1'b0;
    enableLota = 1'b0;
    finishedDelta = 1'b0;
    finishedRho = 1'b0;
    finishedPi = 1'b0;
    finishedChi = 1'b0;
    finishedLota = 1'b0;
    outputDelta = '0;
    outputRho = '0;
    outputPi = '0;
    outputChi = '0;
    outputLota = '0;

    if(enable)
        begin
            enableDelta = 1'b1;
            nState = sDelta;

        end
    end

sDelta:begin
    if(finishedDelta)
        enableRho = 1'b1;
        nState = sRho;
        enableDelta = 1'b0;

    end

sRho:begin
    if(finishedRho)
        enablePi = 1'b1;
        nState = sPi;
        enableRho = 1'b0;

    end

sPi:begin
    if(finishedPi)
        enableChi = 1'b1;
        nState = sChi;
        enablePi = 1'b0;

    end

end
```

```

sChi:begin
    if(finishedChi)
        enableChi = 1'b1;
        nState = sLota;
        enableChi = 1'b0;

    end

sLota:begin
    if(finishedLota)
        enableLota = 1'b1;
        nState = waiting;
        enableLota = 1'b0;
        finalOutput = outputLota;
        outArr = finalOutput;
        finished = 1'b1;

    end

    default:nState = waiting;
endcase

end

endmodule

```

Implementation 2

```

module sha3 #(parameter [6:0] width = 7'b0000010)(input logic [1599:0] arr,input logic
clk,n_rst,enable,output logic [1599:0] outArr,output logic finished);

    logic [1599:0] outputDelta, outputRho, outputPi, outputChi, outputLota, finalOutput;
    logic
enableState,enableDelta,enableRho,enablePi,enableChi,enableLota,finishedDelta,finishe
dRho,finishedPi,finishedChi,finishedLota;

    enum { waiting,sMatrix,sDelta,sRho,sPi,sChi,sLota} pState,nState;

```

```

    delta
d0(.arr(arr),.outArr(outputDelta),.clk(clk),.n_rst(n_rst),.enable(enableDelta),.finished(fin
ishedDelta),.width(width));
    rho
r0(.arr(outputDelta),.outArr(outputRho),.clk(clk),.n_rst(n_rst),.enable(enableRho),.finis
hed(finishedRho),.width(width));
    pi p0(.arr(outputRho),.outArr(outputPi),.clk(clk),.n_rst(n_rst),.enable(enablePi),.finis
hed(finishedPi),.width(width));
    chi
c0(.arr(outputPi),.outArr(outputChi),.clk(clk),.n_rst(n_rst),.enable(enableChi),.finished(f
inishedChi),.width(width));
    lota
l0(.arr(outputChi),.outArr(outputLota),.clk(clk),.n_rst(n_rst),.enable(enableLota),.finishe
d(finishedLota));

```

```

    state_mat
s0(.bitString(arr),.state_matrix(state_matrix),.clk(clk),.n_rst(n_rst),.enable(enableState));

```

```

always_ff@(posedge clk,negedge n_rst)
begin
    if(!n_rst)
    begin

    end
    else
    begin

    pState <= nState;

    end
end

```

```

always_comb
begin

    unique case(pState)

        waiting:begin
            enableDelta = 1'b0;
            enableState = 1'b0;
            enableRho = 1'b0;
            enablePi = 1'b0;
            enableChi = 1'b0;
            enableLota = 1'b0;
            finishedDelta = 1'b0;
            finishedRho = 1'b0;
            finishedPi = 1'b0;
            finishedChi = 1'b0;
            finishedLota = 1'b0;
            outputDelta = '0;

```

```

outputRho = '0;
outputPi = '0;
outputChi = '0;
outputLota = '0;

if(enable)
begin
    enableState = 1'b1;
    nState = sMatrix;

end
end

sMatrix:begin
    nState = sDelta;
    enableDelta = 1'b1;

end

sDelta:begin
    if(finishedDelta)
        enableRho = 1'b1;
        nState = sRho;
        enableDelta = 1'b0;

end

sRho:begin
    if(finishedRho)
        enablePi = 1'b1;
        nState = sPi;
        enableRho = 1'b0;

end

sPi:begin
    if(finishedPi)
        enableChi = 1'b1;
        nState = sChi;
        enablePi = 1'b0;

end

sChi:begin
    if(finishedChi)
        enableChi = 1'b1;
        nState = sLota;
        enableChi = 1'b0;

end

```

```
sLota:begin
    if(finishedLota)
        enableLota = 1'b1;
        nState = waiting;
        enableLota = 1'b0;
        finalOutput = outputLota;
        outArr = finalOutput;
        finished = 1'b1;

    end

    default:nState = waiting;
endcase

end

endmodule
```